What Moves the Ceiling on Automated Audit: from General Code to Scientific Code, Results and Data

Zhaohe Dong [1]

Abstract

Audit benchmarks grade an automated auditor against a hidden reference, but the evidence an auditor sees is often equally consistent with the program and with that reference. In such cases recall minus false-flag rate is bounded by how well the evidence separates defective from correct programs. The bound is a classical fact about tests, yet audit evaluations cannot observe it, because a benchmark fixes a single contract. We make it observable. From HumanEval+ and MBPP+ tasks we build pairs of programs, checked by execution, that differ only on a behaviour the task leaves open; we grade auditors from two vendors against a contract drawn at random and vary how that contract is revealed. On 41 held-out units with the contract withheld, auditors flag the program that departs from the benchmark's behaviour 23.8 points more often than the one that follows it. Solutions written from the task alone almost always follow the benchmark's behaviour as well, so what the benchmark rewards is agreement with the default reading. One added sentence stating the missing requirement raises verified diagnosis by 76.8 points. Among 150 natural hidden-test failures, two model classifiers agree that the task's words single out the expected value in only 34. Three earlier preregistered studies of scientific code, results and data fit the same distinction: false flags on correct scientific code halve when the task names the deliverable, and recomputation catches fabricated results where code and inputs determine them. All held-out analyses were preregistered.

1. Introduction

MBPP+ problem 102 (Liu et al., 2023) asks for a function that converts a snake-case string to camel case and gives three ordinary examples. A generated function splits on underscores, capitalises each word and passes all three. The hidden suite then passes `____python_program` and expects `____PythonProgram`; the function returns `PythonProgram`. The task says nothing about how to handle leading underscores. The benchmark treats this program as defective, so an automated auditor that reads and passes it has missed a defect.

Models increasingly check what other models write. Audit benchmarks score them as in this example, against a hidden reference. We argue that these scores conflate two kinds of case. In some, the evidence available to the auditor determines the correct behaviour, and a miss is the auditor's failure. In others, such as the example above, the evidence is equally consistent with the program and the reference. No auditor, whatever its capacity, can tell them apart without being told which behaviour is wanted. A "detection" in these cases amounts to agreement with the benchmark's convention; it would be a false accusation had the convention gone the other way.

This distinction is classical. For any flagging rule, recall minus false-flag rate is bounded by the total variation between the evidence a defective and a correct program produce. The best attainable recall at a given false-flag rate is the Neyman–Pearson power of that evidence (Section 2). Audit evaluations have lacked a way to observe this distinction because a benchmark fixes one contract and hides the counterfactual. Our experiment constructs that counterfactual (Section 3). For each unit, we take a real benchmark task that leaves one behaviour open and build two programs that differ only there. We grade the auditor against a contract drawn independently of what it sees, then reveal the contract in different ways. Execution validates the programs, oracles and boundaries of the open behaviour; two independent models judge whether both rules are admissible.

Findings. Across 41 held-out units and auditors from two vendors at two reasoning budgets:

- With the contract withheld, flagging cannot track which program is wrong: the same readings are scored under both contracts, so recall equals the false-flag rate (17.4%) by construction. Which program is flagged is

[1]Independent researcher. Correspondence to: Zhaohe Dong <dongzhaohe321418@gmail.com>.

Preprint. November 8, 2026.

telling. The one that departs from the benchmark's behaviour is flagged 23.8 points [10.4, 37.2] more often than the one that follows it, and diagnosed 11.6 points [5.2, 18.0] more often (the benchmark-convention advantage).

- Solutions written from the task alone follow the benchmark's behaviour on these units almost always. The convention the benchmark rewards is the default reading, and rewarding it would be a false accusation had the task's author meant the other rule.
- Adding the missing requirement as one ordinary sentence raises verified diagnosis by 76.8 points [68.3, 84.8] over the task alone; once the contract is stated, 2.1% of correct programs are flagged.
- When told only that the task leaves the choice open, the two vendors respond differently: one passes almost every program, while the other asks for clarification.
- On positive controls, where the task's words rule out one program, the same auditors separate the programs without being told (recall minus false-flag rate 94.2 points).

Among natural hidden-test failures (Section 4), fewer than a quarter are ones that both classifiers agree the task's words determine. Three earlier preregistered studies of scientific code, results and data (Section 5) are consistent with the same distinction, although none of them varied the information an auditor receives in the way our experiment does.

Contributions. (i) A randomised-contract design that makes the information limit of an audit observable and separates convention agreement, unjustified accusation, requests for clarification and verified diagnosis, with execution-validated programs and oracles. (ii) Preregistered evidence that, on such cases, auditors side with the benchmark's default reading, and that one added sentence stating the requirement turns that agreement into verified diagnosis. (iii) A bridge to natural audits showing how hidden-test recall decomposes into determined and undetermined failures, and an interpretation of scientific code, results and data audits using the same distinction between capacity and information misses. Code, records and preregistrations are released.

2. The ceiling is a property of the evidence

Setting. An audit case consists of a program X, the evidence E shown to the auditor (the task text, the visible tests, any added text, and anything the auditor can compute from these, including by executing X), and a hidden contract H that defines the ground truth. The program is defective, $D = 1$, if it violates H. An auditor is any measurable, possibly randomised, map from E to a flag; we write $\phi(e) = \Pr(\text{flag} \mid E = e) \in [0, 1]$. Sampling more readings, using a larger model, reasoning longer or executing the supplied code changes ϕ but not the joint law of (E, D). Write P_1 and P_0 for the laws of E given $D = 1$ and $D = 0$.

Proposition 1 (Evidence ceiling)**.** *For every auditor ϕ,*

$$\text{TPR}(\phi) - \text{FPR}(\phi) \;\le\; \text{TV}(P_1, P_0),$$

and for every $\alpha \in [0, 1]$, $\max\{\text{TPR}(\phi) : \text{FPR}(\phi) \le \alpha\}$ is attained by a likelihood-ratio test of E, so the receiver operating characteristic of the likelihood ratio is the ceiling for all auditors that see E.

Proof. $\text{TPR} - \text{FPR} = \int \phi\,(p_1 - p_0)\,d\mu \le \int_{p_1 > p_0} (p_1 - p_0)\,d\mu = \text{TV}(P_1, P_0)$ for densities p_1, p_0 with respect to a dominating measure μ, since $0 \le \phi \le 1$. The second statement is the Neyman–Pearson lemma. □

Corollary 1 (Indistinguishable component)**.** *Suppose the cases split into components and, on a component K, E has the same law for defective and correct programs. Then every auditor has $\text{TPR}_K = \text{FPR}_K$. If a share π of the defects lies in such components, recall at component false-flag rate at most α is at most $(1 - \pi) + \pi\alpha$.*

Both statements are textbook facts about hypothesis testing (the Neyman–Pearson lemma and the total-variation bound on any test's power minus its size; Neyman & Pearson, 1933). We claim no novelty for them. The component bound requires a false-flag constraint on the component itself. Under an overall false-flag rate α alone, with q the share of correct programs in the component, the bound is only $(1 - \pi) + \pi \min(1, \alpha/q)$. We use these statements to separate two causes of missed defects that audit evaluations conflate. Misses on distinguishable cases are *capacity* misses: some auditor with the same evidence attains higher recall at the same false-flag rate. Misses on indistinguishable cases are *information* misses: there, any auditor buys recall only with an equal rise in false flags, and no auditor does better at a fixed false-flag rate without different evidence. By definition, only a change in E moves the ceiling; the empirical questions are how far real auditors sit below it and which changes in E matter.

Contract randomisation makes the component observable. A natural benchmark fixes the hidden contract to its reference behaviour. Indistinguishability is therefore counterfactual: we cannot see the auditor's response under the other admissible contract. We construct this counterfactual using a discrepancy unit, an input class on which two behaviours, A and B, are both admissible under the task text and the visible tests. We build two programs that differ only on that class, X_A and X_B, and draw $H \in \{A, B\}$ independently of everything the auditor sees.

Lemma 1. *If E depends on (X, H) only through X and H is independent of X with $\Pr(H = A) = \frac{1}{2}$, then $P_1 = P_0$ on the unit, so every auditor's recall equals its false-flag rate, whatever its capacity.*

Proof. With X uniform over $\{X_A, X_B\}$, $D = 1$ exactly on (X_A, B) and (X_B, A) and $D = 0$ on (X_A, A) and (X_B, B). Each event has probability $\frac{1}{4}$, and the law of E given each depends only on X, so $P_1 = P_0 = \frac{1}{2}Q_{X_A} + \frac{1}{2}Q_{X_B}$, where Q_x is the law of E given $X = x$. □

Revealing H in the evidence (stating the requirement) breaks the premise of Lemma 1; text that draws attention to the input class, or lists both behaviours without choosing, does not. The experiment in Section 3 measures how far real auditors sit below the ceiling once the requirement is revealed and whether additional computation helps more before or after the reveal. It also measures how much of the recall on natural benchmarks reflects agreement with the benchmark's convention on indistinguishable units. Those detections would be false flags had the convention gone the other way. The propositions leave these three questions open.

What we do not claim. The ceiling is relative to the evidence, which includes what a model brings with it. A model that has memorised a benchmark's reference sees more than the task text. Contract randomisation neutralises this on the constructed units because the memorised convention is equally likely to be wrong. On natural units we cannot observe the counterfactual contract. Section 4 therefore reports which failures the task's words determine, as judged by two models and checked against what independent programmers do. This describes a benchmark; it does not estimate an information-limited share. A task's logical ambiguity is not the same as equality of the two evidence laws, and nothing here rules out computation helping on natural programs.

3. A randomised-contract audit experiment

Section 2 states that computation cannot repair a miss on an indistinguishable case. In a natural benchmark, we never observe the counterfactual contract. We construct it for each experimental unit: a real benchmark task that leaves one behaviour open. We write two programs that differ only in that behaviour and draw the contract used to grade the auditor independently of what it sees. We then either reveal or withhold the contract.

3.1. Construction

Contracts first. For each of the 542 HumanEval+ and MBPP+ problems (Liu et al., 2023), a proposer model identifies one kind of valid input for which the task text and visible tests leave the behaviour open. It states two rules as requirements a task author could add: *B*, the benchmark reference's behaviour, and *A*, an equally specific alternative (for example, "if several sublists are equally short, return the first" versus "…the last"). Appendix A groups the registered units by the kind of behaviour left open (ties, readings of a phrase, Unicode conventions, repeated items, numeric types, degenerate inputs). Two vendors then implement each rule from the task and the rule alone, using the same prompt; no implementation sees the reference. Auditors read one vendor's A and B implementations, which therefore share provenance and style. The vendor alternates across units.

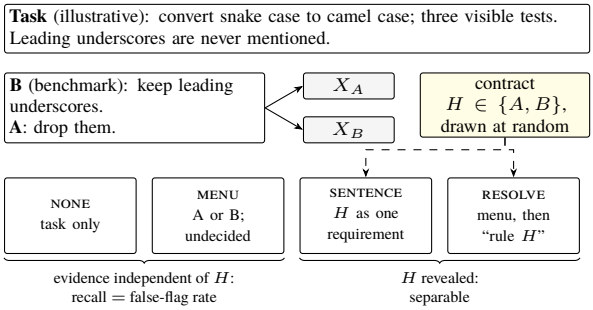


Figure 1. An illustrative unit. Two programs differ only on a behaviour the task leaves open; the contract the auditor is graded against is drawn at random, then withheld or stated. A diagnosis counts only with a witness input on which the program and the contract's oracle disagree under the benchmark's comparator.

Admission by execution. An independent model writes a domain predicate `valid` and 16–24 probe inputs; the proposer writes a class predicate `in_class`. The checks cover the hidden suite, the proposer's inputs, the probes, up to 300 seeded stress mutations (unusual whitespace, combining marks and other Unicode, complex values equal to integers, negative indices, extreme and cancelling floating-point values, empty and duplicated collections) and up to 40 inputs from a red-team model. That model sees all implementations and both predicates and tries to break the admission conditions. Admission requires both B implementations to agree with the reference on every valid input in these sets. Among these valid inputs, all four implementations must agree with the reference outside the class; inside it, the two A implementations must agree. A and B must differ on at least two valid class inputs, and every implementation must pass the visible tests. Each stage is attempted once per problem; problems without a first-attempt primary unit received one second, independent attempt, fixed before any held-out audit. These checks reject, for example, a "correct" B program whose integer summation silently became floating point.

Judging without human raters. Two models that neither proposed the rules nor serve as auditors (Claude Opus, GPT-6.1-sol) judge whether the inputs are valid and each rule is compatible with the task's words and visible tests. They are blind to which rule is the benchmark's and are instructed that a missing example is not permission. The primary population comprises admitted units for which both judges accept the domain and both rules. We call these *execution-validated, two-model-compatible contract pairs*. They are not human-validated ambiguities, and we make no claim about how common such ambiguity is.

Positive controls. The same pipeline yields units whose evidence distinguishes the programs by asking the proposer for a plausible *misreading* that passes the visible tests but that the task's words rule out. Both judges must call the misreading incompatible and the required rule compatible.

3.2. Arms, auditors and scoring

Each open unit is audited under six arms, defined by the text appended to the task: nothing (NONE); both rules as a numbered menu stating that the task does not say which applies (MENU); the same menu plus "the function must follow rule (k)", once for each rule (RESOLVE-A, RESOLVE-B); and one rule stated as an ordinary requirement with no menu, once for each rule (SENTENCE-A, SENTENCE-B). A *reading* is one audit call; a *cell* is a unit, program, arm and auditor configuration, read twice; a *configuration* is one auditor at one reasoning budget. Menu order is fixed per unit and independent of the contract. GPT-5.6-terra and Claude Sonnet 5.5 each audit at a low and a high reasoning budget (SENTENCE arms: low only), using the shipped rulebook and two readings per cell (4,528 readings in all, including the positive controls).

Under contract X, program X is correct and the other is defective. In NONE and MENU, the evidence does not depend on the contract, so we score the same readings under both contracts. Lemma 1 then forces flag recall to equal the false-flag rate; we report this equality as a check. A cell flags if either reading reports a blocking finding. A diagnosis is *verified* only if a flagging reading supplies a witness call whose arguments have the benchmark's own types, that satisfies `valid` and lies in the class. On that call the two contract oracles must differ, the program must disagree with the contract's oracle (the reference for B, the other vendor's implementation for A) under the benchmark's own comparator, and the value the auditor says the call should return must equal the oracle's. All of this is checked by execution. A unit's value is the mean over configurations; intervals are problem-level bootstraps. We test the false-flag budget after resolution (10%) separately with a betting confidence bound (Waudby-Smith & Ramdas, 2024), which does not gate the other claims.

3.3. Results

The held-out population contains 41 units from 41 problems (34 from the first construction attempt and 7 from the second) and 3,280 readings; the positive-control population contains 78 units. All hypotheses, intervals and the order in which claims are tested were registered before the first held-out call.

Adding the requirement turns agreement into diagnosis. One ordinary sentence stating the contract raises verified diagnosis by 76.8 points [68.3, 84.8] over the task alone (E6-5). A menu of both rules followed by a choice does about as well: 91.2%, against 0.0% with the undecided menu (E6-1: +91.2 [82.9, 97.6]). Because the undecided menu itself suppresses flagging, E6-5 is the contrast that measures what a stated requirement adds to the plain task. The menu doubles as an attention control. It is longer than the sentence, names the input class and states both rules, yet it yields almost no verified diagnosis; the gain comes from knowing which rule holds, not from being pointed at the right place. Once the contract is stated, the flag rate on correct programs is 2.1%, with a betting upper bound of 11.3% against the registered 10% budget (E6-1b, not met).

Without the contract, auditors side with the benchmark. In the NONE arm recall equals the false-flag rate (17.4%) by construction, since the same readings are scored under both contracts. What carries information is which program gets flagged. The program that departs from the benchmark's behaviour is flagged 23.8 points [10.4, 37.2] more often than the one that follows it; the frozen code computes this, though the registration does not list it as a hypothesis. Verified diagnoses, likewise, are 11.6 points [5.2, 18.0] more likely when the contract is the benchmark's rule (E6-2). Whether this reflects memory of the benchmark or the most natural reading of the task we cannot say, and for an evaluation it makes no difference. Solutions written from the task alone follow the benchmark's rule on these units almost always (Appendix A). A hidden-test "detection" here therefore credits agreement with the default reading, and it would be a false accusation had the task's author meant the other rule.

Reasoning budget. After the contract is stated, the high budget changes verified diagnosis by 0.6 points (bootstrap [0.0, 1.8]; betting $[-10.9, 12.0]$). The registered equivalence claim (margin 10) is inconclusive: near the ceiling and at this sample size the data cannot distinguish no effect from a moderate one, and we make no claim either way.

The vendors respond differently to an open question. When told that the task leaves the choice open, GPT passes 93.3% of readings and Claude asks for clarification in

87.8% (Figure 2c). Neither flags. Both behaviours are defensible, but only one is visible to a benchmark that scores flag or pass.

Positive controls. Where the task's words rule out one program, the same auditors separate the two programs without any added text: flag recall minus false-flag rate is 94.2 points [90.7, 97.1] (E6-4). The flat NONE arm of the open units therefore reflects evidence that does not decide, rather than an auditor that cannot read.

4. What natural audit failures are made of

The randomised experiment isolates the information limit on constructed units. We ask how much of an ordinary audit benchmark is subject to that limit. We use 150 programs written by two generator models for HumanEval+ and MBPP+ problems. Each program passes its visible tests but fails the hidden suite and is paired with a test-passing control (Liu et al., 2023).

Most hidden-test failures are not determined by the task. Two models classify each program's first failing input, blind to which output is the benchmark's (order randomised by evidence). The categories are: the input is outside the task's domain, only the reference's output is acceptable (DETERMINED), both are (OPEN), only the program's is (AT ODDS), or neither. Identical evidence is pooled so that it cannot receive two labels. Of the 150 failures, 34 are DETERMINED, 26 OPEN, 37 AT ODDS, 21 out of domain and 32 disputed (Cohen's $\kappa = 0.78$ on the 43 development items). Only on 34 of the 150 (23%) do both classifiers agree that the task's words single out the value the hidden suite expects; on 63 they agree that the words leave it open or point the other way.

Other solutions mostly side with the "defective" program. A behavioural check avoids classifier judgement. For each failure, two generators wrote twelve solutions from the task alone; we run the 1,710 of the 1,800 that pass the visible tests on the same first failing input. They return the defective program's result 1,080 times and the reference's 559 times (71 neither); for 93 failures most solutions side with the defective program, for 46 with the reference. The solutions come from the same two generators that wrote the failing programs, so this is partly a model agreeing with itself; still, on the inputs that define these programs as defective, the benchmark's expected value is the minority reading among solutions written from the task. The behaviour also checks the classification (Figure 3b): where the classifiers judge that only the program's output fits the words (AT ODDS), 90% of independent solutions return it; where they judge the reference's value determined, solutions split almost evenly, which is consistent with defects the generators commonly share.

What auditors verify. With the original task text, four readings per program and two auditors, verified diagnosis on the held-out failures is 25.0% on DETERMINED failures and 19.0% on OPEN ones. The registered contrast (H6) is 6.0 points [−20.2, 28.3] and is not supported: on these failures the class of the first failing input does not predict whether an auditor verifies a defect (the development estimate, from 13 items, was 50 points). The witnesses tell a more specific story. Of the 19 held-out programs with a verified diagnosis, 17 have at least one witness that both classifiers label DETERMINED: an auditor that verifies a defect in an OPEN failure usually does so through a different input, on which the words do decide. Hidden-test recall on such a benchmark therefore mixes two things: on behaviour the words determine it measures the auditor, and elsewhere it measures at best agreement with a convention the task does not state. The classification concerns the first failing input only, so a program may also have determined defects elsewhere.

Clarifying the missing requirement. For the 13 failures that remain OPEN after reclassification and whose added sentences pass a neutrality recheck, we appended the missing requirement as one sentence, a length-matched placebo, or a pointer naming the input class without resolving it. The 9 held-out items (7 problems) provide a small replication of Section 3: the clause raises verified diagnosis by 88.9 points [62.5, 100.0] over the placebo (H1) and by 61.1 points [38.9, 85.0] over the pointer (H5), and the placebo does nothing (H4). Whether the clause also raises flags on correct programs could not be settled at this size (H2: −5.6 points [−33.3, 22.2]).

5. Transfer: scientific code, results and data

Section 2 separates capacity misses, where the evidence decides the case but the auditor does not use it, from information misses, where the evidence does not decide the case. Three preregistered studies of scientific work are consistent with this distinction. They preceded the experiment above, are reported in full in Appendix B, and use ground truth that no model supplies; none of them randomised the contract, so they illustrate the distinction rather than test it.

Code: what the auditor is told it is reviewing. SciCode (Tian et al., 2024) splits research problems into steps with tests written by scientists. We label generated steps that pass the tests correct and those that fail defective (the tests can themselves reject correct code; Hu et al., 2026) (81 defective, 150 correct instances). The auditor sees the step's text and code but no tests. Given the whole problem as the task, a cross-vendor auditor at eight readings flagged 87.7% [72.7, 98.6] of defective and 68.7% [56.6, 79.7] of correct

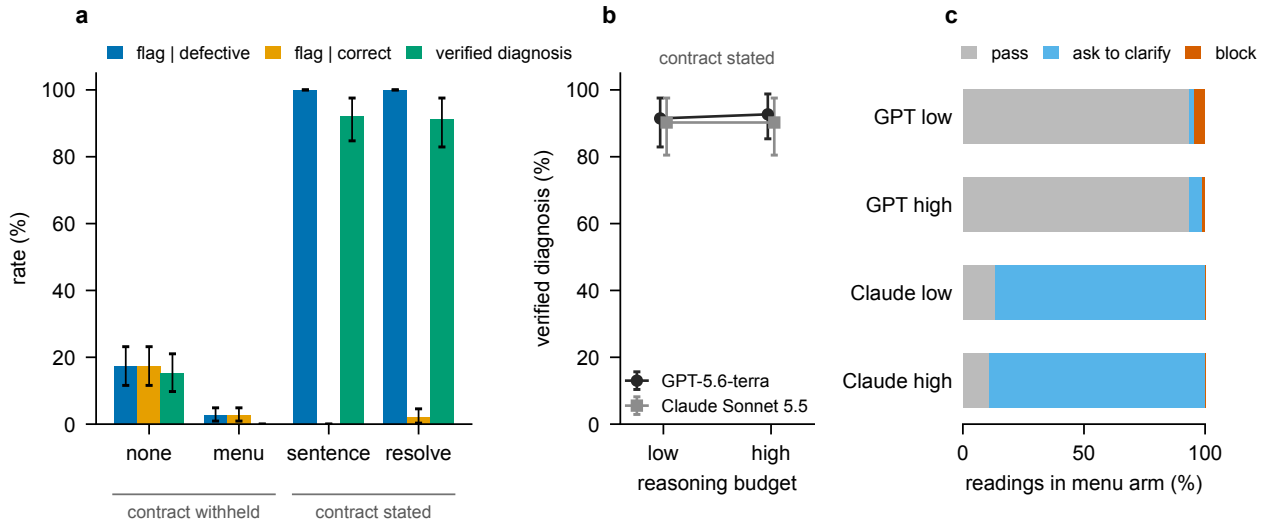


Figure 2. Randomised-contract experiment, held-out units. (a) Flag rate on defective and on correct programs and verified diagnosis, pooled over the four auditor configurations (sentence: low budget only). With the contract withheld the two flag rates are equal by construction (Lemma 1); stating it, as a choice between the two rules (resolve) or as one ordinary requirement (sentence), separates them. (b) Verified diagnosis after resolution at the low and high reasoning budget. (c) What auditors do when told that the task leaves the choice open (menu arm). Bars and intervals: problem-level means with 95% bootstrap intervals.

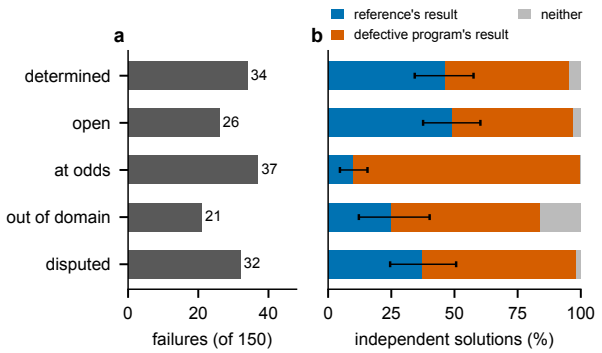


Figure 3. Natural hidden-test failures (150 programs that pass their visible tests). (a) Class of the first failing input, as two classifiers judge it from the task's words and visible tests, with identical evidence pooled. (b) What independent solutions written from the task alone (and passing the visible tests) return on that input, by class; the interval is an item-level bootstrap for the share returning the reference's result.

steps. Some objections concerned a single step's failure to implement the whole problem. A second registered arm changed only the task text to name the step as the deliverable; the program bytes were identical. Flags on correct code fell by 32.7 points [22.4, 42.8], and flags on defective code by 14.8 points [2.7, 28.6]. What the auditor was told it was reviewing moved its operating point more than reading more often did.

Results: where the evidence decides, computation helps. For 74 correct SciCode programs we built a run log, a results file and a report in which every number cites its source. We injected either a reporting fault (the artefacts disagree) or a fabrication written consistently into all of them. A deterministic provenance check flagged all 49 reporting faults and none of the 25 fabrications, whose artefacts all agreed. The model auditor flagged 23 of the 25, usually by recomputing the result from the code and inputs (exact McNemar $p = 2.4 \times 10^{-7}$). Re-execution uses the same information and flagged 70 of the 74 faulty items. It missed four whose change fell inside the default numerical tolerance. The evidence here determines the right value, so a miss is a capacity miss. More capacity, through a model that computes or simply running the code, removes most of them.

Data: where the documentation carries the information. We gave each of twelve public regression tables a data card. Each table was either left clean or given one injected fault (mixed units, negated values, duplicated rows, a shuffled target, swapped columns, sentinel values, rounding). We held out eight tables (266 items) and combined a validator written mechanically from the card with a model auditor (GPT or Claude, with a shell on the full table or reading 120 rows). Adding the model raised correct localisation, scored by code against the injection manifest, by 2.3 points [0.4, 4.8] and flags on clean items by 16.0 points [3.2, 31.4]. Neither the full table nor tool use helped. In an unsandboxed pilot, one auditor searched the disk and the web for the original files. Every subsequent reading ran in an operating-

system sandbox. Even there, 57% of that auditor's held-out readings attempted out-of-scope access; these attempts were logged and returned nothing. An earlier study on four tables, at a clean-flag budget fixed in advance, found a larger gain in findings naming the faulty column (10.7 points [6.0, 15.9]) over a simpler validator. In both studies, where the documentation already states what valid data look like, a check written from it finds most of the faults that it and the model find together.

6. Related work

Critics and repeated reading. Trained critics catch more bugs in model-written code than unaided humans, but also hallucinate some (McAleese et al., 2024). Aggregating review passes raises review F1 on real pull requests (Zeng et al., 2025), and coverage under repeated sampling grows roughly log-linearly in the number of samples (Brown et al., 2024). Self-correction without external feedback does not reliably help (Huang et al., 2024), and critique benchmarks score critics against reference answers (Lin et al., 2024). These studies vary the auditor while holding the evidence fixed. We hold the auditor fixed and vary the evidence; on the cases we construct, changing the evidence moves performance far more than anything we varied about the auditor.

What the hidden suite decides. EvalPlus rebuilt HumanEval and MBPP with larger suites (Liu et al., 2023). Models collapse onto one reading of an underspecified task (Richter & Papadakis, 2026), and generated tests err mostly in the expected value (Xiong et al., 2023). Generated oracles capture what a program does rather than what was intended (Konstantinou et al., 2024). Label errors in test sets bound and reorder what benchmarks can measure (Northcutt et al., 2021; Gema et al., 2025), which motivates relabelled "platinum" benchmarks (Vendrow et al., 2025); SWE-bench Verified removed, by human annotation, issues whose descriptions were underspecified or whose tests were too specific (OpenAI, 2024); and ambiguous questions in open-domain QA admit several correct answers (Min et al., 2020). In our setting, the reference is not wrong: it is one of several behaviours the task admits. The appropriate correction is to state the contract, rather than relabel or discard the case as those efforts do. We measure the effect of stating it.

Clarification. Code generators can be made to ask clarifying questions when a task is ambiguous (Mu et al., 2024; Wu & Fard, 2025), or to formalise intent as tests confirmed by a user (Lahiri et al., 2022). Abstaining when evidence is insufficient is the classical reject option (Chow, 1970). We study auditing and score a request for clarification as a separate outcome, neither a detection nor a pass. We find that the two vendors' auditors differ sharply in how often they request clarification.

Auditing scientific work. Models detect a minority of errata-grade errors in manuscripts (Son et al., 2025) and of paper–code discrepancies (Baumgärtner & Gurevych, 2026). An auditor of AI-scientist projects detects pitfalls far better with logs and code than with the paper alone (Luo et al., 2025). Cleaning agents miss errors that span rows (Bendinelli et al., 2025). An expert audit of SciCode (Tian et al., 2024) found specifications and tests that reject correct solutions (Hu et al., 2026) and repairs them by rewriting the specification. Our randomised contracts are the experimental counterpart of that repair. They measure how much of an auditor's apparent miss rate a repaired specification would remove.

Statistics. The ceiling is the Neyman–Pearson bound (Neyman & Pearson, 1933). We test the false-flag budget with a betting confidence sequence (Waudby-Smith & Ramdas, 2024), which is valid at any sample size. Preregistration of evaluations is beginning to appear in machine learning (Vaccaro, 2026; Thomas et al., 2026). We use it as a method and do not claim it as a contribution.

7. Discussion and limitations

Interpretation. On units where the task leaves a behaviour open, a hidden-test "detection" is agreement with the benchmark's behaviour, which is also the default reading of the task: auditors flag the program that departs from it far more often, and solutions written from the task alone follow it (Section 3). Adding one sentence that states the requirement turns that agreement into verified diagnosis; we could not resolve whether a larger reasoning budget changes the result once it is stated. Natural failures are observational and classified by models, but they point the same way: both classifiers agree that the task's words settle fewer than a quarter of failures, and other solutions mostly behave like the "defective" program (Section 4). The earlier scientific studies are consistent with the distinction without testing it (Section 5): naming the deliverable cut false flags on correct scientific code from 69% to 36%, and where code and inputs determine a reported value, recomputation catches fabrications that consistency checks cannot.

Practical consequences. Alongside recall, an audit benchmark should report which of its cases the task determines. It should not count agreement with an unstated convention as detection. An auditor that is unsure whether a behaviour is required should be scored for asking, not only for flagging or passing. For deployment, an auditor's value on such behaviour depends on whether the requirement is written down; discovering requirements that nobody wrote down, which we did not measure, is the harder problem.

Limitations. *Constructed units.* The randomised-contract units are execution-validated; two models, rather than people, judge them admissible. We make no claim about how often such ambiguity occurs. Agreement between two models that a rule is compatible with a task is weaker than expert agreement. *Benchmarks.* All randomised-contract and natural-failure units come from HumanEval+ and MBPP+, short functions with small specifications. Whether the size of the convention advantage carries over to larger programs, where more behaviour is unstated, remains untested. *Auditors.* We tested two vendors' models at two reasoning budgets with one rulebook. Other auditors, agent scaffolds or execution-based test generation may sit closer to the ceiling before the reveal, although Lemma 1 bounds what any of them can do on the unrevealed arms. *Verification.* A verified diagnosis requires a witness that the benchmark's own comparator confirms and two judges accept. We do not credit diagnoses stated only in prose, which can understate auditors whose findings are correct but unexecutable. *Natural failures.* Models classify the natural failures, with Cohen's κ of 0.78 on development items. The behavioural check uses solutions from two generators only. *Data.* The data study injects one fault per item into twelve public tables; its conclusion is specific to faults that documentation describes. *Discovering requirements.* Our registered arms state the requirement or list both candidates. In a registered follow-up (E8, Appendix A), an auditor asked to list unspecified behaviours named the open behaviour in 97.6% of held-out readings but also named the decided behaviour of positive controls in 80.8%, so discovery comes with little selectivity and returns the question to the task's author. *Earlier studies* of the shipped configuration (Appendix B) are reported as registered and are not re-analysed here.

Impact Statement

This paper concerns how much automated auditors of model-written code and data should be trusted. It argues against reading a hidden-test recall figure as auditor capability when the task does not determine the expected behaviour, and against counting agreement with a benchmark's convention as detection. A risk is that the constructed units are used to tune auditors toward a benchmark's conventions; we release them with both contracts for that reason. All data are public benchmarks and datasets under permissive licences; no human subjects were involved.

References

Baumgärtner, T. and Gurevych, I. SciCoQA: Quality assurance for scientific paper–code alignment. *arXiv preprint arXiv:2601.12910*, 2026.

Bendinelli, T., Dox, A., and Holz, C. Exploring LLM agents for cleaning tabular machine learning datasets. *arXiv preprint arXiv:2503.06664*, 2025.

Brooks, T. F., Pope, D. S., and Marcolini, M. A. Airfoil self-noise and prediction. Reference Publication 1218, NASA, 1989.

Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., and Mirhoseini, A. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

Chow, C. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970. doi: 10.1109/TIT.1970.1054406.

Gema, A. P., Leang, J. O. J., Hong, G., Devoto, A., Mancino, A. C. M., Saxena, R., He, X., Zhao, Y., Du, X., Ghasemi Madani, M. R., Barale, C., McHardy, R., Harris, J., Kaddour, J., van Krieken, E., and Minervini, P. Are we done with MMLU? In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 5069–5096, 2025. doi: 10.18653/v1/2025.naacl-long.262.

Hamidieh, K. A data-driven statistical model for predicting the critical temperature of a superconductor. *Computational Materials Science*, 154:346–354, 2018. doi: 10.1016/j.commatsci.2018.07.052.

Hu, S., Huang, L., Deng, Y., and Chen, K. SciCode-Verified: How benchmark defects underestimated the scientific-coding ability of language models. *arXiv preprint arXiv:2608.04975*, 2026.

Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.

Konstantinou, M., Degiovanni, R., and Papadakis, M. Do LLMs generate test oracles that capture the actual or the expected program behaviour? *arXiv preprint arXiv:2410.21136*, 2024.

Lahiri, S. K., Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., Musuvathi, M., Choudhury, P., von Veh, C., Inala, J. P., Wang, C., and Gao, J. Interactive code generation via test-driven user-intent formalization. *arXiv preprint arXiv:2208.05950*, 2022.

Lin, Z., Gou, Z., Liang, T., Luo, R., Liu, H., and Yang, Y. CriticBench: Benchmarking LLMs for critique-correct reasoning. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 1552–1587, 2024. doi: 10.18653/v1/2024.findings-acl.91.

Liu, J., Xia, C. S., Wang, Y., and Zhang, L. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2305.01210.

Luo, Z., Kasirzadeh, A., and Shah, N. B. The more you automate, the less you see: Hidden pitfalls of AI scientist systems. *arXiv preprint arXiv:2509.08713*, 2025.

McAleese, N., Pokorny, R. M., Cerón Uribe, J. F., Nitishinskaya, E., Trebacz, M., and Leike, J. LLM critics help catch LLM bugs. *arXiv preprint arXiv:2407.00215*, 2024.

Min, S., Michael, J., Hajishirzi, H., and Zettlemoyer, L. AmbigQA: Answering ambiguous open-domain questions. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 5783–5797, 2020. doi: 10.18653/v1/2020.emnlp-main.466.

Mu, F., Shi, L., Wang, S., Yu, Z., Zhang, B., Wang, C., Liu, S., and Wang, Q. ClarifyGPT: A framework for enhancing LLM-based code generation via requirements clarification. *Proceedings of the ACM on Software Engineering*, 1(FSE):2332–2354, 2024. doi: 10.1145/3660810.

Neyman, J. and Pearson, E. S. On the problem of the most efficient tests of statistical hypotheses. *Philosophical Transactions of the Royal Society of London. Series A*, 231:289–337, 1933. doi: 10.1098/rsta.1933.0009.

Northcutt, C. G., Athalye, A., and Mueller, J. Pervasive label errors in test sets destabilize machine learning benchmarks. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. arXiv:2103.14749.

OpenAI. Introducing SWE-bench verified. https://openai.com/index/introducing-swe-bench-verified/, August 2024.

Richter, C. and Papadakis, M. Underspecification does not imply incoherence: The risks of semantic collapse in coding models. *arXiv preprint arXiv:2607.01953*, 2026.

Son, G., Hong, J., Fan, H., Nam, H., Ko, H., Lim, S., Song, J., Choi, J., Paulo, G., Yu, Y., and Biderman, S. When AI co-scientists fail: SPOT, a benchmark for automated verification of scientific research. *arXiv preprint arXiv:2505.11855*, 2025.

Thomas, M., Gligorić, K., and Shah, N. B. Mitigating LLM-based p-hacking by preregistering for the next LLM. *arXiv preprint arXiv:2606.27687*, 2026.

Tian, M., Gao, L., Zhang, S. D., Chen, X., Fan, C., Guo, X., Haas, R., Ji, P., Krongchon, K., Li, Y., Liu, S., Luo, D., Ma, Y., Tong, H., Trinh, K., Tian, C., Wang, Z., Wu, B., Xiong, Y., Yin, S., Zhu, M., Lieret, K., Lu, Y., Liu, G., Du, Y., Tao, T., Press, O., Callan, J., Huerta, E., and Peng, H. SciCode: A research coding benchmark curated by scientists. *arXiv preprint arXiv:2407.13168*, 2024.

Tüfekci, P. Prediction of full load electrical power output of a base load operated combined cycle power plant using machine learning methods. *International Journal of Electrical Power & Energy Systems*, 60:126–140, 2014. doi: 10.1016/j.ijepes.2014.02.027.

Vaccaro, M. Preregistration for experiments with AI agents. *arXiv preprint arXiv:2606.11217*, 2026.

Vendrow, J., Vendrow, E., Beery, S., and Madry, A. Do large language model benchmarks test reliability? *arXiv preprint arXiv:2502.03461*, 2025.

Waudby-Smith, I. and Ramdas, A. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 86(1):1–27, 2024. doi: 10.1093/jrsssb/qkad009.

Wu, J. J. and Fard, F. H. HumanEvalComm: Benchmarking the communication competence of code generation for LLMs and LLM agent. *ACM Transactions on Software Engineering and Methodology*, 2025. doi: 10.1145/3715109.

Xiong, W., Guo, Y., and Chen, H. The program testing ability of large language models for code. *arXiv preprint arXiv:2310.05727*, 2023.

Yeh, I.-C. Modeling of strength of high-performance concrete using artificial neural networks. *Cement and Concrete Research*, 28(12):1797–1808, 1998. doi: 10.1016/S0008-8846(98)00165-3.

Zeng, Z., Shi, R., Han, K., Li, Y., Sun, K., Wang, Y., Yu, Z., Xie, R., Ye, W., and Zhang, S. SWR-Bench: Assessing LLM performance in real-world code review comment generation. *arXiv preprint arXiv:2509.01494*, 2025.

A. The randomised-contract experiment in detail

A.1. Construction funnel

Table 1 follows every problem from proposal to the registered population. Each stage was attempted once per problem and kind; a failed attempt consumes it. Second-attempt (open2) proposals were made, before any held-out audit, only for problems without a first-attempt primary unit under the admission then in force, and the analysis takes a second-attempt unit only for a problem without a first-attempt one.

Table 1. E6 construction funnel (development / held-out). *Pre-admission*: both implementations of each rule pass the visible tests and differ on the proposer's inputs. *Admitted*: admission v4.3 on the hidden suite, proposer inputs, probes and stress inputs inside the interface. *Red team*: no violation among the red-team model's interface inputs, at least one such input. *Judges*: both judges accept the domain and both rules (controls: rule A incompatible, rule B compatible). *Registered*: one unit per problem.

	Open, 1st attempt	Open, 2nd attempt	Positive controls
Problems	136 / 406	122 / 374	136 / 406
Proposal parsed	135 / 394	119 / 362	134 / 399
Pre-admission	55 / 140	39 / 112	70 / 221
Admitted (v4.3)	25 / 81	18 / 53	38 / 105
Red team passed	22 / 66	14 / 37	35 / 82
Judges passed	13 / 34	5 / 12	30 / 78
Registered	13 / 34	3 / 7	30 / 78

The most common admission failures were disagreement with the reference outside the class (an implementation of rule A that changed behaviour beyond the open input class) and a rule-B implementation that disagreed with the reference somewhere (for example, integer summation that silently became floating point). Admission is an operational guarantee: on every registered input inside the programs' interface on which the reference runs, both rule-B programs return the reference's result, both rule-A programs return it outside the class and agree with each other inside it, and the two rules differ on at least two valid class inputs. It does not establish whole-program correctness.

A.2. What the open behaviours are

Table 2 groups the 57 registered open units (16 development, 41 held out) by the kind of behaviour the task leaves open; the grouping is ours, made from the rules before any held-out audit. Most units turn on a choice a careful programmer makes without noticing: which of several tied items to return, whether repeated items count once, what a letter or a space is outside ASCII, or whether `True` is an integer.

Table 2. Kinds of open behaviour among the registered open units (development / held out), with one example each (rules shortened).

Kind	Units	Example (task: rule A / rule B)
Ties and precedence	3 / 11	MBPP+ 457, sublist of minimum length: return the last / the first of several tied sublists
Reading of a phrase or structure	2 / 12	MBPP+ 576, "is the second list a sublist": consecutive positions / any order-preserving positions
Unicode and text conventions	5 / 6	MBPP+ 624, upper-case a string: ß becomes the capital sharp s (U+1E9E) / SS
Repeated items	3 / 5	MBPP+ 629, even numbers of a list: each distinct value once / every occurrence
Numeric types and booleans	2 / 4	HumanEval+ 22, filter integers: keep / drop Boolean values
Degenerate inputs	1 / 3	MBPP+ 93, power: 0^0 raises `ValueError` / returns 1

What independent solutions do. For every registered unit we ran the solutions that two generators (Claude Haiku 4.5 and GPT-5.6-terra, six samples each) wrote from the task alone and that pass the visible tests on the unit's class inputs where the two rules differ. Of the classified solutions, 98.4% follow the benchmark's rule B on development units (187 of 190; 3 follow A) and 89.1% on held-out units (415 of 466; 32 follow A and 19 neither; a further 12 cannot be classified because no class input separates the rules); on 34 of the 41 held-out units every classified solution follows B. These are construction records, computed before any held-out audit. One generator is also one of the auditors, so the agreement between what

auditors flag and what generators write is partly agreement of a model with itself.

Can an auditor find the open behaviour by itself? (E8.) On every registered unit, Claude Sonnet 5.5 (low budget) was shown the task, the visible tests and one of the two programs and asked for up to six behaviours that the task leaves unspecified, as questions to the task author. Claude Opus 5.5, shown both rules in a random order and not told which is the benchmark's, judged whether any question targets the behaviour on which the rules differ. On the 41 held-out open units it did in 97.6% of readings [93.8, 100]; on the 78 held-out positive controls, where the task's words decide that behaviour, it also did in 80.8% [75.0, 86.4]. The difference, 16.8 points [10.5, 23.2] (problem-paired bootstrap), shows some selectivity, but an auditor asked to look still names four in five decided behaviours as unspecified: its questions are a broad list of the usual edge cases, and telling which of them the task leaves open is left largely to the task's author. E8 was registered after a development pilot (96.9% against 88.3%), uses one vendor for both roles and relies on a model judge; all 238 held-out readings completed.

A.3. Inputs, interface and red team

The registered input set of a unit is the hidden suite, the proposer's class and non-class inputs, 16–24 probes from a model that did not propose the rules, up to 300 seeded type-guided stress inputs (unusual whitespace, combining marks and other Unicode, complex values equal to integers, negative indices, extreme and cancelling floating-point values, empty, singleton and duplicated collections), and up to 40 inputs from a red-team model shown the task, the reference, both rules, both predicates and all four implementations. Every check, and every credit in scoring, is restricted to inputs whose arguments have the types the benchmark's own inputs use at that position, element types included; behaviour on other types belongs to neither contract. A batch that stalls or exceeds a memory cap is re-executed with one killed child process per call, so that calls the per-call alarm cannot interrupt become timeouts instead of stalling admission.

A.4. Judging, auditing and scoring

Two models that neither proposed the rules nor audit judge, blind to which rule is the benchmark's and with the rule order randomised, whether the domain is valid and whether each rule is compatible with the task's words and visible tests; a missing example is not permission. Each registered open unit is read under six arms by four configurations (two vendors, two reasoning budgets; the sentence arms at the low budget only), two readings per cell: 80 readings per unit. Positive controls are read in the unrevealed arm only, 16 readings per unit. A reading flags if it reports a blocking finding; a cell flags if either reading does. A diagnosis is verified only if a flagging reading supplies a witness call inside the interface that satisfies the domain predicate, lies in the class, and on which the two contract oracles differ, the program disagrees with the contract's oracle under the benchmark's own comparator, and the value the auditor says the call should return equals the oracle's; every step is executed, none is judged by a model. Sensitivity analyses replace the domain predicate by the reference's own domain and credit witnesses outside the class.

A.5. Statistics

Unit values average over configurations; intervals are problem-level percentile bootstraps (10,000 draws). The false-flag budget after resolution (E6-1b, 10%) and the equivalence of the two reasoning budgets (E6-3, margin 10 points) use betting confidence bounds (Waudby-Smith & Ramdas, 2024), which are valid at any sample size but wide at ours. Hypotheses are tested in a registered order (E6-1, E6-2, E6-5, E6-3), each claimed only if all earlier ones hold; positive controls form a separate family (E6-4). Missing cells are reported with bounds that set excluded units to the worst and best values.

A.6. Protocol history

The protocol was revised six times on development units before any held-out audit, each time after an adversarial methods review by a model from another vendor; every version, its admissions and its reasons are archived. The revisions added stress inputs (after a review found admitted programs that disagreed with the reference on unseen inputs), replaced a too-narrow class predicate by the proposer's, corrected a sign error in the betting bound, separated the false-flag budget from the ordered family, added the second construction attempt, added the red-team stage, restricted every check and credit to the interface, and rescored earlier development witness records under the final rule (some had been scored by an earlier witness stage without the executable domain, class and oracle checks; development results in this paper use the rescored records). The registration file lists each change with its date and the development results before and after it.

B. Earlier studies of one shipped audit configuration

C. The system under study

Every auditor reading in this paper is taken through one shipped system, CrossAudit. Every auditor prompt is built by the product's own prompt builder, and every family but one is read through the product's own audit call; the exception, `gpt-6-astra`, receives the same prompt bytes through a direct call. We describe the system first because its design fixes what the later sections can and cannot vary. We state its structure only here and quote no measurement of it. Its limits are the subject of Section D onward.

Separation of roles. A generator model writes an increment (a set of files) into a Git repository. An auditor model then reads the committed bytes and returns findings, each graded BLOCKER or ADVISORY. The auditor must come from a different vendor from the generator: the audit refuses to run when the configuration names the same vendor for both roles, and when the generator's vendor is not declared, the receipt records that independence could not be asserted. The same-vendor arms below run with the generator's vendor undeclared, deliberately. The premise is that a model reviewing its own vendor's output is an interested reader. The product takes that premise as given, and Section D puts one consequence of it to the test: whether a same-vendor auditor catches more or less.

Two tiers of evidence. Before the auditor reads anything, a deterministic check layer runs registered checks over the same committed bytes. A check's finding is *verified*: code produced it and code can reproduce it. A model's finding is *raised*: it is a reading, not yet reproduced. The checks are grouped into named profiles, so strictness is a setting the project chooses:

- `general`, the default: structured files parse, every local path a file lists as an input or dependency exists, and relative links and leftover placeholders are reported as advisory.
- `science`: results carry a schema; quantities carry units; a result that reports convergence must meet its own threshold; every input a result depends on is listed, exists and is cited at an exact revision (*provenance*); and every number a report states is annotated with the file and quoted span it came from, which a check matches against the cited bytes (*number→source*).
- `research`: the general profile's checks plus a citation check under which a report may cite only sources it fetched through a governed tool.

The science profile follows one rule throughout, and the rule is the one Section E tests: *the model names its evidence, and code verifies it.* A number with no source line, a unit that disagrees with its schema or an input that does not exist fails without any model's opinion.

Computed verdicts. The verdict is derived by code from the evidence, in a fixed order. A failed deterministic check blocks, whatever the model says. A model BLOCKER with no deterministic support either blocks or escalates to a person, according to a project setting. A blocked increment goes back to the generator for a bounded number of revision rounds, three by default. Each revision passes a repair guard before it is committed: an edit outside the audited scope is refused, and a new catch-all handler, a deleted assertion or a skipped test is flagged to the auditor for the next round.

Receipts. A passing round produces a receipt that binds the verdict to the commit that was audited, the commit of the rulebook the auditor was given, and a digest over every finding with its tier. Every model call is written to a ledger with its prompt hash and cost. A verification command recomputes the digest from the recorded evidence, so a receipt can be checked by someone who does not trust the operator.

What the rest of the paper measures. Section D, with Appendices F and G, measures the model tier on its own. There, an increment counts as flagged when the auditor grades any finding BLOCKER, and the general profile's checks run as in the product but decides no recall or false-positive outcome. The model tier is the part of the system whose limit is least known, since a deterministic check catches exactly what it was written to catch. Section E measures the model tier on scientific code, and sets it beside deterministic verification on reported results and on data, asking what each flags that the other does not.

D. The ceiling on general-purpose code

Substrate and ground truth. HumanEval+ and MBPP+ (Liu et al., 2023): Python functions with an executable hidden test suite. One frozen generator (`claude-haiku-4-5`) was shown each specification and its visible tests and wrote one

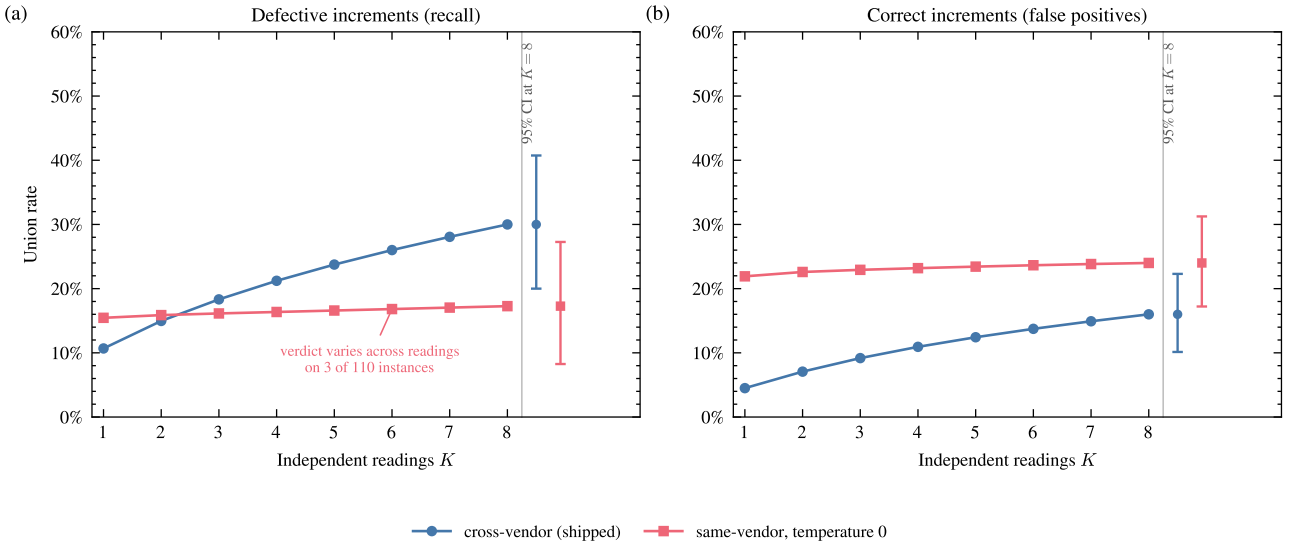


Figure 4. **Returns to repeated reading diminish short of complete.** Union rate against the number of independent readings K, for two auditor families. (a) Recall on defective instances; (b) false positives on correct instances. Bars at the right of each panel give the 95% problem-cluster bootstrap interval of each series at $K = 8$, dodged in x so overlapping intervals stay separable. The cross-vendor curve does not meet the preregistered flattening bar (its last-step gain is 1.93 points [1.14, 2.78] against a bar of 1.0) and is not described as saturating. The same-vendor curve meets it, at 0.23 points, but that route is sent temperature 0 and its verdict varies across readings on only 3 of 110 defective instances under this configuration, so its flatness is not evidence of exhausted headroom; whether it reflects the sampling or the model is not separated here (Section G.2).

solution per problem, over two passes of 540 problems. A candidate that passes the visible suite and fails the hidden one is *defective*; one that passes both is *correct*. Recall is measured on all 110 defective instances (56 problems) and false positives on 150 correct ones drawn by seed. The auditor sees the specification, the visible tests and the candidate, never the hidden suite. Because the defective stratum already survived the visible tests, recall here is the fraction of that harder subset caught, not of all defects a model writes (Appendix F).

Estimand and statistics. Union recall at K: the fraction of defective instances blocked (any finding graded BLOCKER) by at least one of K independent readings, averaged exactly over all size-K subsets of the readings. The same union on correct instances is the false-positive rate, always reported beside it (Table 3 collects the primary quantities). Intervals are 95% problem-cluster percentile bootstraps; paired contrasts carry exact McNemar and cluster sign-flip tests. The shipped cross-vendor route (`gpt-5.6-terra`) sends no sampling parameter; same-vendor routes are sent temperature 0, a difference that bounds every cross-family union contrast (Section G.2).

Repeated reading. Union recall rises from 10.7% at one reading to 30.0% [20.0, 40.7] at eight, at 16.0% [10.1, 22.3] on correct code (Figure 4). The eighth reading still adds 1.93 points [1.14, 2.78], above the 1.0-point flattening bar registered in advance, so we report diminishing returns, not saturation. Twenty readings pooled across three families reach 48.2% [36.7, 60.0] at 36.0% [28.2, 44.2] on correct code: pooling buys coverage at its own price.

Which model reads. Replacing the auditor with a stronger same-vendor model lowered union recall at eight readings by 26.4 points [−37.3, −15.6] (exact McNemar $p = 4.2 \times 10^{-7}$; cluster sign-flip $p = 2.0 \times 10^{-5}$), a preregistered primary. This is not a measure of auditing ability. The stronger model also flags far less correct code, 3.3% [0.7, 6.7] against 16.0%, and under an exploratory any-finding rule the sign reverses: 59.1% [47.3, 70.6] recall at 34.0% [26.5, 41.7] on correct code, against 34.5% [23.6, 45.5] at 19.3% [12.8, 26.3] (Figure 6 in Appendix G). At one reading, before the differently sampled unions grow apart, the gap is only −10.1 points. A high-reasoning cross-vendor model reaches 32.7% [20.7, 45.0] at 10.7% [5.9, 16.1] with four readings, against the shipped route's 30.0% at 16.0% with eight, but no paired contrast establishes both advantages together.

The rulebook. We added one sentence to the auditor's rulebook: *find what the visible tests do not cover*. At eight readings on all 110 defective instances, it raised union recall from 30.0% to 60.9% [48.6, 72.5], +30.9 points [19.1, 43.1] (preregistered), at a price of 36.7% [28.5, 44.8] false positives against 16.0%. An earlier revision experiment asked whether the changed

Table 3. Selected quantities of the general-code studies (Section D and Appendix G), with the population each is measured on and the label its source report gives it. Values are percentages or percentage points. Intervals are 95% problem-cluster percentile bootstraps unless the label names another method. The coverage simulation of §F.3 measures single-rate intervals only; the contrasts and the category-conditioned shares have no measured coverage. The two third-rater rows rest on labels from a rater that agreed with itself on 7 of 11 repeated items (6 problems). The correct-code row counts disagreement on inputs extracted from the findings; of the 20 flagged instances, 9 agreed on every extracted input and 9 yielded no valid input, which can be neither confirmed nor refuted. Every row is read from a committed record by `figures/make_table1.py`.

quantity	population	value	interval	label
Union recall, shipped route, $K = 8$	110 defect, 56 problems	30.0%	[20.0, 40.7]	preregistered
Union false positives, shipped, $K = 8$	150 correct, 143 problems	16.0%	[10.1, 22.3]	preregistered
Pooled union, 3 families	110 defect	48.2%	[36.7, 60.0]	preregistered
its false positives	150 correct	36.0%	[28.2, 44.2]	derived
Gain from the eighth reading	110 defect	+1.93	[1.14, 2.78]	bar 1.0, not met
Same-vendor stronger − shipped, recall	110 defect	−26.4	[−37.3, −15.6]	preregistered
its false-positive difference	150 correct	−12.7	[−19.3, −6.1]	prereg.; 3.3% vs 16.0%
Rulebook rule − shipped, recall	110 defect	+30.9	[+19.1, +43.1]	preregistered
its false-positive cost	150 correct	+20.7	[+12.8, +28.8]	preregistered
Rulebook rule, flags	56 one-revision defect	+26.8	[+13.6, +40.4]	exploratory
its cost on correct code	56 one-revision correct	+12.5	[−3.6, +26.8]	unconditional, approx.
isolating contrast, pass rate	112 one-revision, 96 problems	+5.4	[−0.9, +12.1]	registered, failed
`astra` − shipped, common $K = 4$	110 defect, 56 problems	+11.5	[+2.9, +21.3]	exploratory
its false-positive difference	150 correct	−0.3	[−3.8, +3.4]	exploratory
Undetermined, sheet groups	121 entries, 110 instances	+19.8	[+1.2, +38.1]	registered method
one definition of caught	57 against 53	+23.3	[+2.0, +43.6]	post hoc
Undetermined share, 3-family residual	44 of 57	77.2%	[62.1, 91.1]	post hoc
6-family residual	25 of 32	78.1%	n/a	post hoc
Correct-code flags, reference disagrees	2 of 20 flagged	10.0%	[0.0, 26.3]	preregistered
Clarified − original, diagnosis	32 selected, 19 problems	+28.1	[+7.7, +51.4]	one-signed; exact $p = 0.0625$

flags make code pass. There, at one reading on 56 defective instances (41 problems), flags rose from 10 to 25, +26.8 points [13.6, 40.4] (exploratory; exact McNemar $p = 0.0003$, cluster sign-flip $p = 0.0009$). On correct code they rose by 12.5 points, an interval [−3.6, +26.8] that includes zero. An improvement in outcomes was not established: on the contrast that isolates the rule, the hidden-suite pass rate after one revision moved +5.4 points [−0.9, +12.1]. The sentence changes what the auditor writes; whether that makes code pass is not established.

What is missed. Fifty-seven of the 110 defective instances were flagged by no reading of the first three families. Two model raters, one of them the agent that ran this programme, agree post hoc that in 44 of them (77.2% [62.1, 91.1]) the specification's prose does not entail the value the hidden suite expects on the first failing input, as in the example that opens this paper. The category is broad: it covers prose that is silent, prose that allows the program's value, and prose that determines a value the suite contradicts. That reverses an earlier classification of the same instances as unexercised edges. Being undetermined is not enough to be missed: reading six families (five distinct models, one of them also read with the added rulebook sentence) at forty readings shrinks the never-flagged set to 32 of 110, and 43 of the 68 instances both raters call undetermined are flagged by some reading. The undetermined share of what remains is similar, 25 of 32 (78.1%), but the 32 are a subset of the 57, so this is a description, not a test of stability. Of twelve MBPP tasks an outside paper names as ambiguous, incomplete or contradictory (Richter & Papadakis, 2026), 11 are in our defect population, and our frozen labels call 9 of them undetermined (post hoc). An attempt to build a population whose specifications determine the defect by construction did not deliver that property, so this remains a post hoc description (Appendix G.5).

What follows. A union recall figure is contingent on the false-positive rate it cost, the model and sampling of the route, the rulebook, the decision rule, and what the benchmark's specifications determine. Nine further studies, reported in Appendix G.5 with what each licenses, include a specification-rewriting intervention that raised adjudicated diagnosis only where the added rule did not wholly contradict the hidden suite, and a second substrate that closed without a quotable number.

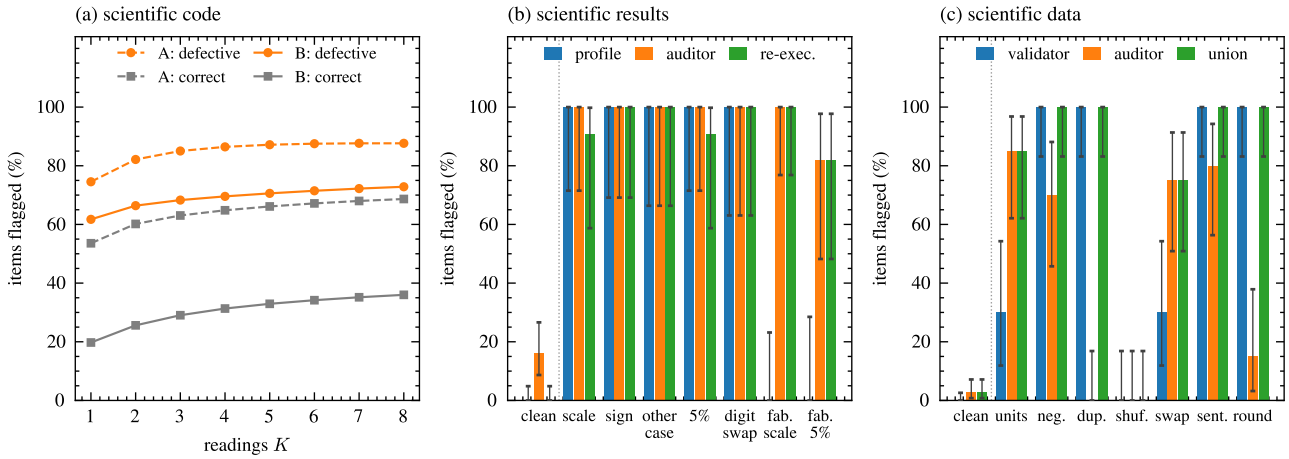


Figure 5. **What each tier flags.** (a) Scientific code: the shipped auditor's union flag rate against the number of readings, on defective (orange) and correct (grey) instances, in arm A, whose task gave the whole problem (dashed), and arm B, whose task was reframed around the step (solid). (b) Scientific results and (c) scientific data, by fault type, with clean items leftmost: the deterministic tier (blue: the `science` profile in b, the documentation-derived validator in c), the model auditor at four readings (orange), and re-execution (b) or the union of the two (c) in green. In (c): mixed units, negated values, duplicated rows, shuffled target, swapped columns, -999 sentinel, rounding. The model auditor's flag is any model BLOCKER in the union of readings; the profile's is a hard failure, the validator's any fired check, re-execution's a value outside `np.allclose` defaults. Bars carry exact binomial 95% intervals on the item counts (supplementary; they ignore clustering).

E. Auditing AI for science

A scientist would hand an auditor three jobs: check the code that computes a result, check that a reported result follows from that code, and check the data an analysis starts from. We ran one preregistered study for each, with ground truth no model supplies: scientists' numeric tests for code, re-execution for results, and injected faults for data. Every item is audited through the product's own audit call and shipped rulebook, with the same flag rule as above. Construction, gates, amendments and full results are in Appendix H.

E.1. Scientific code

Design. SciCode (Tian et al., 2024) decomposes research problems from 16 subfields into steps with numeric tests written by scientists. The same frozen generator wrote every step three times, each sample building on its own earlier steps. An instance is correct if it passes the step's tests and defective if not; a step enters only if some sample passes it, which excluded 184 of 336 evaluated steps. This gives 81 defective instances (30 problems) and 150 correct ones (53 problems). Labels are per step, but the auditor sees the whole chain, and 51 of the defective and 49 of the correct instances have a failing earlier step (post hoc description). SciCode has no visible/hidden split, so the auditor sees the step's prose, background, header and code, and no tests. The shipped cross-vendor auditor and the generator's own model each read every instance eight times. After the registered arm's outcomes, a second registered arm changed only the task text, naming the step's function as the deliverable instead of opening with the whole problem; the program bytes are identical (Appendix H.1).

Results. What the task named as the deliverable changed the flag rate substantially (Figure 5a). With the whole problem as the task, the shipped auditor flagged 87.7% [72.7, 98.6] of defective and 68.7% [56.6, 79.7] of correct instances at eight readings, and some of its objections to correct code were that one step did not implement the whole problem. With the task reframed around the step, flags on correct code fell to 36.0% [26.4, 45.6], a paired change of -32.7 points [-42.8, -22.4]. Recall fell too, to 72.8% [59.3, 87.3], a change of -14.8 points [-28.6, -2.7], though the cluster sign-flip test for it gives $p = 0.0625$. The change is the whole reframing (the step named, the problem description moved last and marked as context, earlier functions exempted), and the arms ran in sequence. The generator's own model's union recall was 46.9 points lower than the shipped auditor's in each arm ([-63.0, -30.7] and [-61.5, -34.4]; a route comparison), while flagging 27.3% [18.4, 37.3] and 18.0% [11.5, 25.6] of correct code, respectively.

At one reading in the reframed arm, the shipped auditor flags 61.7% [47.9, 75.9] of defective and 19.8% [12.9, 27.2] of correct instances: it discriminates, modestly. We read a seeded sample of 20 flagged correct instances whose earlier steps

all pass, taking the first BLOCKER of the first flagged reading of each; four of the twenty concern earlier functions that the reframed task excluded from the deliverable. They range from real divergences between code and step text, through contradictions inside the benchmark's own instructions, to tests too weak to separate outputs; which flags point at defects was not established. An independent expert audit of SciCode found self-contradictory specifications, non-reproducible gold answers and over-tight tolerances that reject correct solutions (Hu et al., 2026), so some instances labelled defective here may be correct code the benchmark rejects. Of the 81 defective instances, 7 and 19 escaped all sixteen readings of the two families in the two arms. Both raters judged 4 of 7 (two problems; cluster interval [0.0, 80.0]; $\kappa = -0.235$) and 13 of 19 (eight problems; [28.6, 93.5]; $\kappa = 0.587$) undetermined by the prose, too few problems and too little agreement for a stable share. The registered test that this share exceeds the general-code residual's 77.2% is killed in both arms by its own interval rule.

E.2. Scientific results

Design. Each of 74 correct SciCode instances (36 problems) whose output on one seeded test case is a real scalar or a short real array becomes a results increment: the program, the test's inputs, a script that runs one on the other and writes a run log, a `results.json` citing each value's log line, and a report whose numbers carry the product's number→source annotations. Each yields a clean item and a faulty one. Five fault types are *reporting* faults, where the artefacts disagree with each other or with the log; two are *fabrications*, a $\times 10^3$ slip or a 5% error written consistently into log, results and report. The model auditor reads each item four times with the shipped `science` profile on; the profile's own verdict comes from the same call; and re-execution compares the output with the report (Appendix H.2).

Results. The deterministic profile flags all 49 reporting faults and none of the 25 fabrications, because in those everything agrees with everything else (Figure 5b). The model auditor flags 23 of the 25 fabrications, 21 with a sound finding about the fault (post hoc), usually by working the result out from the code, for example evaluating a KL divergence to 1 where the report says 1000; the other two rest on wrong calculations. All 23 discordant items go the model's way (exact McNemar $p = 2.4 \times 10^{-7}$). On reporting faults its rate is of the combined workflow, since its prompt carried the profile's findings. Re-execution misses four faults whose change to a near-zero output falls within `np.allclose`'s default absolute tolerance. Setting those four aside (post hoc), re-execution flags 70 of 70 faulty items at 0 of 70 clean, against the model auditor's 68 of 70 at 12 of 70. On the registered population the two flag 70 and 72 of 74 faulty items, and eight of the model's twelve clean-item flags identify real construction defects, so this subset favours re-execution for injected numerical faults without making it the better check in general. The model auditor also flags 12 of 74 clean items. Read one by one (post hoc), 8 correctly identify defects in our construction (a provenance sentence false where the benchmark passes inputs as call literals; a unit label contradicting documented units); 4 rest on wrong calculations or a syntax error that is not there.

E.3. Scientific data

Design. Four CC BY 4.0 physical-science tables (Yeh, 1998; Brooks et al., 1989; Tüfekci, 2014; Hamidieh, 2018); 280 items of 120 rows with a data card giving each column's meaning and unit: 140 clean, and 140 carrying one of seven faults (mixed units, negated values, duplicated rows, a shuffled target, swapped columns, a -999 sentinel, rounding). Both model families read each item four times, against a documentation-derived validator written from the card and frozen before any item existed (Appendix H.3).

Results. The two tiers flag overlapping but different faulty items (Figure 5c). The validator flags 92 of 140 faulty items (65.7% [57.9, 73.6]) and no clean item; the shipped auditor 65 of 140 (46.4% [38.6, 54.3]) at 4 of 140 clean items (2.9% [0.7, 5.7]); their union 112 (80.0% [73.6, 86.4]) at the same 4. These intervals resample items within dataset; four datasets are too few to resample, so no interval carries between-dataset variation. The auditor's 4 clean flags and 4 of its faulty-item flags are requests for fuller column descriptions on one card; counting only findings about the injected fault (post hoc), the auditor flags 61 and the union 110. Of the 20 faulty items the auditor flags and the validator misses, 18 carry a finding about the injected fault itself (post hoc), mixed units or swapped columns; in one it argued that three atoms spanning a 135 pm radius range cannot have a mean radius of 1.6 pm, the injected cell being that value converted to ångströms. It flagged no shuffled target and no duplicated rows (0 of 20 each), and did worse than the validator on every fault the validator was written for. The generator's own model flags 67 of 140 clean items, and for at least 48 of those (a post hoc lower bound) one finding states a row count, field count, missing column or truncation the file contradicts. The validator omits one range the concrete documentation states and uses a list of continuous columns the card does not give.

Held-out naming of faults at a fixed budget. The counts above rest on flags, and their diagnosis counts are post hoc. A

further preregistered study asked, on 280 new items built the same way, whether the model auditor names faults the validator misses at a clean-flag budget fixed in advance. The endpoint was scored by code, not by a model: a flagging finding counts when it names the injected column (both columns for a swap; duplication for duplicated rows), read from the injection manifest. The rule is lexical and is not a measure of correct diagnosis. Operating points were fixed on the items above: the smallest vote threshold over four readings with a clean-flag rate of at most 10% (one reading for the shipped auditor, all four for the generator's own model). On the held-out items the validator meets the rule on 90 of 140 faulty items (64.3% [56.4, 72.1]) at no clean flag, and the validator with the shipped auditor 105 (75.0% [67.6, 81.9]) at 9 of 140 clean flags (6.4% [2.8, 10.5]): $+10.7$ points [6.0, 15.9], 15 items one way and none the other (exact McNemar $p = 6.1 \times 10^{-5}$), the registered primary. At its own operating point the generator's own model meets it on 40 at 14 clean flags, 12.1 points [4.1, 20.3] fewer than the shipped auditor (a route comparison). The rule also counts some findings about other things. On the study agent's unblinded post hoc reading of the 15 items the union adds, 14 locate the fault in the right columns ($+10.0$ points [5.4, 15.1]) and 11 also explain it ($+7.9$ [3.7, 12.5]; Appendix H.3).

F. Method in full

F.1. Substrate, population and ground truth

Every general-code result (Section D and Appendix G) is measured on one substrate: HumanEval+ and MBPP+ (EvalPlus). These are Python functions whose median task is 41 words of prose and 6 lines of reference code, each with an executable hidden test suite. An interpreter therefore decides every stratum, recall denominator and false-positive population, and no model judges a recall or false-positive outcome. The one outcome defined by adjudication, correct diagnosis in the clarification study, is judged jointly by two models and is labelled so where it appears. The benchmark's base suite is shown to the generator as the *visible* suite, and its plus suite is held back as the *hidden* one. We also built and audited a second substrate, the standard-library half of BigCodeBench. It closed without a quotable number when its preregistered criterion was not met on a third run, and nothing from it is reported here.

How the population was built. The corpus has 542 problems. Two, `HumanEval/32` and `Mbpp/590`, were excluded by name before generation because their own reference solutions fail their own hidden suites in this harness, which leaves 540. One frozen generator, `claude-haiku-4-5-20251001`, was shown each specification and its visible suite and wrote one solution per problem. A candidate that passes the visible suite and fails the hidden one is a *defective* instance; one that passes both we call *correct*; one that fails the visible suite is discarded from every recall and false-positive population. The defect stratum is small, about 10% of solutions. Under a rule registered before the second pass, and depending only on execution outcomes, we therefore repeated generation over the same 540 problems until there were at least 100 defective instances or three passes had run. It stopped at two passes: 56 and 54 defective, 455 and 455 correct, 29 and 31 discarded. An instance is therefore a (pass, problem) pair, and one problem can contribute two. Recall is measured on all 110 defective instances, which come from 56 problems. False positives are measured on 150 of the 910 correct instances, from 143 problems; that sample was drawn once from a fixed seed before any auditor ran. Seven of the 110 defective instances fail by not terminating rather than by a failed assertion, and the registered population keeps them.

Two consequences of these definitions hold throughout. First, *correct* means *passes the hidden suite*. It is a benchmark label produced by an interpreter, not a guarantee that the program meets its specification, and a suite can be incomplete or wrong. Since one of this paper's results (§G.4) is that a specification often fails to determine what the hidden suite asks, we cannot treat passing that suite as correctness in the ordinary sense, and a *false positive* is likewise a flag on test-passing code rather than a demonstrated mistake by the auditor. Second, the defect population is not a sample of model-written code: it is the residue that already survived the visible tests, which is by construction the harder part. A recall figure here is the fraction of that residue an auditor catches, never the fraction of all defects a model writes.

F.2. The auditor and the estimand

The auditor reads the specification, the visible tests and the candidate, and returns findings with severities; an increment is blocked when any finding is graded BLOCKER. It never sees the hidden suite. The estimand throughout is union recall at K: the fraction of defective instances blocked by at least one of K independent readings, averaged exactly over all $\binom{K_{\max}}{K}$ subsets rather than by sampling. The matching quantity on the correct stratum is the union false-positive rate, and a union recall figure is reported only beside the false-positive rate it cost.

Five auditor routes appear below, with the short labels the figures use: the shipped cross-vendor auditor `gpt-5.6-terra`

(`cross`); the same-vendor `claude-haiku-4-5` (`self`), the generator's own family and size; the stronger same-vendor `claude-sonnet-4-6` (`self-strong`); `claude-opus-4-8` (`self-frontier`), read at $K = 4$; and `gpt-6-astra` at high reasoning (`astra`), read at $K = 4$. The auditor's constitution, its prompt builder and every route's capability card are in the auditor harness, a separate repository, at the commit named in the provenance file, whose per-reading records carry the hash of every prompt actually sent.

The routes are not sampled alike, and this bounds every cross-family union-of-K contrast in the paper. The cross-vendor route's capability card does not permit a sampling temperature, so no sampling parameter is sent and the route varies between readings. The same-vendor routes permit one, and the layer sends temperature 0, so those routes are nearly deterministic. A union over readings that do not differ cannot grow, so the two families' unions are not equally free to rise with K, and a cross-family difference in union recall confounds auditing ability with how much each union could grow. The asymmetry is a property of the shipped routing layer we measured rather than a design choice. We did not discover it until after the runs, and we report the $K = 1$ contrast beside the $K = 8$ contrast wherever it matters. The asymmetry is the first item in §7.

F.3. Statistics

The unit of observation is the instance and the resampling unit is the problem, because problems recur across generation batches and instances sharing a problem are not independent. The K readings of an instance are repeated measurements of it, not K further independent observations. The bootstrap over problems estimates uncertainty from which tasks the benchmark happens to contain. It does not estimate how much a fresh batch of model calls on the same tasks would move the numbers, which would need replicate runs we did not budget for at every arm.

Every rate carries a 95% percentile bootstrap interval over whole problem clusters (10,000 resamples, seeds fixed in each preregistration). Where the source report gives a Wilson interval, it appears beside the bootstrap, marked as too narrow. Paired contrasts carry an exact McNemar test and a cluster-level sign-flip permutation test. Where every discordant pair points one way, the percentile bootstrap's bound at zero is an artefact, and an unconditional interval (grid-approximated, unclustered) is quoted instead. The one exception is the clarification study, whose registration fixed the bootstrap for a contrast that turned out one-signed; its interval is reported as registered and read beside its exact cluster test (§G.5). A saturation curve is summarised by a constrained exponential fit, and the fitted asymptote is quoted only when the preregistered flattening bar is met; otherwise it is labelled an extrapolation and the raw union at $K_{\max}$ is the number quoted. Counts of five or fewer are quoted as counts. Each study handles multiplicity as it registered. The study carrying the primary contrasts corrects over every comparison it computed: a Bonferroni family of sixteen with a threshold of 0.00313, which two comparisons clear. Other studies registered a single primary, and the second-vendor rulebook study reports its twenty further quantities without correction.

We measured interval coverage by simulation rather than assuming it. The simulation uses the two strata's actual cluster profiles (110 instances in 56 problems, and 150 in 143) and a within-problem concordance of 0.5, over 400 replicates of 600 resamples each. The table gives the percentile bootstrap's coverage of a nominal 95% rate interval. The Monte Carlo standard error is about 0.011 near 95% coverage and 0.020 at the lowest cell. The simulation covers single rates only, and it uses integer-index percentiles where the analyses interpolate (`analysis/coverage_simulation.py`).

true rate	0.03	0.05	0.10	0.16	0.30	0.48	0.60
defect (56)	0.798	0.870	0.933	0.933	0.953	0.935	0.928
correct (143)	0.888	0.918	0.938	0.933	0.943	0.955	0.965

In these simulations coverage is near nominal at the true rates this paper's headlines sit at and badly anticonservative at small true rates: at a true rate of 0.03 on the defect stratum the simulated interval covers about 0.80, not 0.95. This is evidence of potential undercoverage for analogous intervals, not a measured coverage of any interval quoted here. Intervals on rates below roughly 0.10 may be narrower than they claim. The simulation costs no model calls. Two paired-contrast constructions tried in this programme covered 0.416 and 0.075 where 0.95 was claimed, and neither is used. No sample size was justified in advance by a minimum-detectable-effect calculation.

An interval that contains zero is reported as one that does not exclude zero, and nothing more. No equivalence margin was registered anywhere in this programme and no equivalence test was performed, so none of those intervals is evidence that two conditions are alike or that an intervention did nothing. Where a hypothesis fails its registered criterion we say it

failed, not that the effect is absent.

All analysis of the general-code studies is plain Python 3.13 using only the standard library (`random`, `math`, `itertools`, `csv` and `json`). The bootstrap, the exact McNemar test, the cluster sign-flip permutation and the exact subset enumeration behind every union-at-K figure are implemented in the repository rather than called from a numerical package, so each can be read where it is used and none depends on a third-party version.

F.4. Preregistration and independent review

Who did what. The studies were designed, run, analysed and written up by an AI research agent, Claude (Anthropic; `claude-opus-5` and `claude-fable-5-1`, working in Claude Code), under the direction of the author, who is responsible for the work. Wherever a rating or adjudication below is attributed to the agent, it was that agent. It wrote the rubrics, knew the hypotheses and the earlier labels, and is from the same vendor as the generator. No human rated or adjudicated anything in this paper. Every report was then reviewed by a second model, `gpt-6-astra`, as described below.

Each study that made model calls was preregistered before its first one, fixing its population, hypotheses, primary outcome, intervals, kill conditions, budget and stopping rule, with two exceptions. First, the severity sweep of §G.2 re-grades findings already archived, makes no model calls, and was never registered: every figure it produces is post hoc and exploratory, and none is a confirmatory test. Second, the analysis behind §G.5's third-rating result was registered a day *after* its contrast had been computed and committed, in a document that claimed the opposite. That document is withdrawn in the repository, and the analysis is registered nowhere. Its method was fixed in advance by an earlier amendment, but the disjoint reading, the abstention-excluded secondary and the broken-sheet comparison were not.

Planned departures are numbered amendments, each recording what was known at the time, and in the general-code studies each was committed before the step it governs, with one exception: Amendment 1 of the study that added the `self-strong` and `self-frontier` routes was committed after 30 of its second-draw readings had landed, not before them as it first stated, and that study's Amendment 2 corrects this on the record. In the science studies two were not: A4S-3's Amendment 1 was committed together with the rebuilt items it governs, and A4S-2's Amendment 2 after the two pilot readings it already governed. Both are recorded as errata in the registrations (Appendix I). Deviations discovered only afterwards, such as a leak check or a spend guard implemented more narrowly than registered, are disclosed in the report and are not amendments. Where an amendment followed an interim observation, the results say so and argue whether the change could favour the hypothesis.

Each report was then reviewed by a model from a vendor other than the generator's. The reviewer had read access to the raw records and was asked to re-run the analysis, recompute the intervals independently, and check every sentence against the records. That reviewer is `gpt-6-astra`, which is also one of the auditor families measured in §G.2, one of the two raters behind §G.4's labels, and one of the two adjudicators whose joint labels define the clarification study's outcome. For most claims this is a cross-vendor check on work the reviewer had no part in. For the claims that rank `astra` against the other families, and for the clarification study's adjudication, the reviewer is the subject, and those results should be read as self-assessed until a model that audited nothing in this programme has checked them. For the same reason, the clarification study's Amendment 10 disqualified this model as the study's manipulation-check rater. It kept the model as that study's second adjudicator and its reviewer, however, so the standard was applied unevenly within one study and not applied to the review.

Reviews are archived verbatim, including those that refused approval, and several studies needed many rounds. On one day five reports were refused at once, and clearing those five took thirty-one further rounds. In most of those rounds, what changed was what the reports claimed about their measurements. Most of the later refusals objected to a repair written to satisfy the previous round, not to the study it repaired. A fix written to satisfy a reviewer is itself an unreviewed claim.

The reviewer is a language model, and this paper measures the ceiling on language models reading work they did not produce. That applies to our own instrument. The studies of §G.5 cleared review at rounds 3, 3, 4, 6, 7, 9, 10 and 11, and the ceiling study behind the headlines at 21. In every case the round before the last still returned substantive findings. Some of those findings were defects introduced by the repairs between rounds, so the counts do not estimate a single pass's recall; they show only that no report cleared in fewer than three. *Quotable at round N* means a report survived N passes, not that it was verified, and the marks in this paper should be read as a filter of unknown recall.

Two further points follow. First, measurements did move on review. The severity sweep's first review found its loader

misreading one family's draws and another family's population, and every cross-family figure was recomputed. The clarification study's second review found an instance wrongly excluded, which moved its primary from 9 of 31 to 9 of 32 instances and its contrast from $+29.0$ to $+28.1$ points. And a label count inside a reviewed report was 71 where its data say 67. Once a figure's inputs were right, the reviewers reproduced rates, intervals, subset enumerations and seeds independently and found no arithmetic error, and most refusals were about what the prose claimed. The reviews were reliable on recomputation and unreliable on interpretation, and the two should not be quoted as one assurance. Second, the class of defect this leaves open is the one that looks right to a language model. The clarification study audited the wrong program for 384 readings, because a per-problem field was used where a per-instance one was needed. Every automated gate passed. Each checked that the code was *identical across conditions*, and it was, because every condition showed the same wrong program. The agent found the error while reading an adjudication sheet, by noticing that a displayed program returned exactly what its hidden suite was said to reject. Neither the gates nor a reviewer found it, and we do not know what else of that kind remains.

Every review prompt in this programme stated the claimed result before the reviewer began. We tried to test what that costs, and the attempt failed. One report, quotable at round 6 after five refusals and unchanged since, was dispatched again under a prompt whose own text withheld every figure and every prior verdict and asked for an independent computation first. That prompt's list of binding artefacts then named two files that state those figures, one of them the claims-ledger entry for the result under review. The blinding was not enforced, so the agreement between that pass and the briefed one establishes nothing. What the pass did return is a disagreement count: six substantive findings that six briefed rounds on the same file had not produced. Among them, the report's two groups were two different definitions of *caught* rather than one population with duplicate rows; its rating sheet displays at most three failing inputs while 81 of its 121 entries have more recorded; and a count of changed labels was 71 where the data say 67. The disagreement count is $n = 1$, uncontrolled and unregistered. A seventh round might have found new things under any prompt, and no matched briefed round was run against the same file. We report it as a reason to test blinding properly rather than as a result about blinding. Later first rounds withheld the conclusions from the prompt text, but the report under review stayed among the named artefacts, with an instruction to derive the result before reading it, and that request does not blind the reviewer.

Every study drawn on here has completed that review. One further study was re-run twice and closed without a quotable number; the paper names it only to say what it no longer supports.

G. Results on general-purpose code in full

The claims this paper defends, each with its interval and its label, are listed in `CLAIMS.md`. A claim enters that file only after its report has passed an independent review by a model from a different vendor with access to the raw records, and every section below draws only on such reports. Section G.5 reports eight studies whose reviews took between three and eleven rounds each to reach that point, and states what each does and does not license. The headline results of §G.1–§G.4 come from the ceiling study; where they cite a study from §G.5, they say so.

Rates carry the 95% problem-cluster percentile bootstrap as the primary interval. Problems recur across generation batches, and an interval that treats instances as independent is too narrow. Where the source report gives a Wilson interval, it appears beside the bootstrap. For single rates, simulations of an analogous interval under this clustered design (§F.3) suggest coverage near nominal at the headlines' rates and possible undercoverage below a true rate of about 0.10, under an assumed within-problem correlation. They do not measure the coverage of the intervals reported here, and the paired contrasts and category-conditioned shares have no measured coverage.

Sixteen comparisons were computed in the study that carries the primary contrasts, under one Bonferroni family with a threshold of 0.00313. That correction is applied to p values and never to intervals. Every interval in this paper is an unadjusted 95% interval, so a directed statement resting on an interval alone, with no p value that clears its family, is worded as a direction rather than as a finding. Each p value belongs to the family of the study that computed it. Those in §G.3 are members of that sixteen-member family. The one in §G.2 is the auditor-family study's single registered primary, so there was no family to correct within; it would also clear 0.00313. The clarification study's p values are reported beside a primary registered on its interval, and no correction was registered for them. Where a p value appears below, the exact McNemar value and the cluster-level sign-flip permutation value are both given, because instances within a problem are not independent and only the second respects that.

Table 3 (Section D) collects selected primary quantities with their populations and labels; the prose gives each one where it is used.

Table 4. **Study map, general-purpose code.** Every study reads the same frozen generator's candidates (110 defective instances from 56 problems, 150 correct from 143) unless the population column says otherwise. "Route comparison" marks contrasts that change model and sampling together.

Study	Population	What varies	Main quantity	Status
Repeated reading (§G.1)	110 defective, 150 correct	readings $K = 1$–8; shipped cross-vendor vs same-vendor route	union BLOCKER recall and false positives	preregistered primary
Stronger routes (§G.2)	as above	two stronger same-vendor models: `self-strong` at $K = 8$, `self-frontier` at $K = 4$	union recall of `self-strong` at $K = 8$	preregistered primary; route comparison
Decision-rule sweep (§G.5)	archived findings of the two studies above	four grading rules; no model calls	union recall under each rule	not registered
Rulebook revision (§G.3)	56 defective (41 problems) and 56 correct, one reading	one added rulebook sentence; one revision round	flags on the 56 defective; hidden-suite pass after revision on all 112	outcome contrasts registered; flag contrasts exploratory
Referent rule, second family (§G.5)	as repeated reading, $K = 4$	the same sentence, and a grading rule, on `self-strong`	union BLOCKER recall	preregistered
Referent rule at $K = 8$ (§G.5)	as repeated reading	the same sentence on the shipped auditor	union recall at $K = 8$	preregistered
Residual rating (§G.4)	57 instances no reading of three families flagged	rubric: does the prose entail the expected value	share rated undetermined	rating registered; association post hoc
Outside rater (§G.5)	121 residual and flagged entries	a rater outside the residual's families	share undetermined, missed vs caught	not registered (a named exception)
Specification edit (§G.5)	32 selected residual instances	added rule vs same-length placebo vs original, $K = 4$	jointly adjudicated diagnosis	preregistered
Test validation (§G.5)	frozen generated tests and candidates, including candidates failing their visible tests	two corroboration rules	share of retained failing test applications that are wrong	preregistered; criterion not met
Injected defects (§G.5)	constructed defects and unmodified twins	injected edit	blocks, $K = 8$	descriptive only
Flags on correct code (§G.5)	150 correct, $K = 8$	finding texts kept	share of flagged instances that disagree with the reference on an extracted input	preregistered
Second substrate (§G.5)	BigCodeBench standard-library half	substrate	—	closed; nothing reported

Table 4 lists the general-code studies before their numbers appear: what each read, what it changed, what it measured and how its main quantity was registered.

G.1. Returns to repeated reading diminish short of complete

The shipped cross-vendor auditor reads an increment holistically and returns findings; an increment is blocked if any finding is graded BLOCKER. Over eight independent readings of the same 110 defective instances, union recall rises from 10.7% at one reading to 30.0% [20.0, 40.7] at eight (Figure 4). The eighth reading still adds 1.93 points, cluster [1.14, 2.78], against the flattening bar of 1.0 points registered before the first model call, so the bar was not met. Under exact subset averaging that statistic has a closed form: the number of instances flagged by exactly one of the K readings divided by Kn, here $17/(8 \cdot 110)$. It is a count of singletons, not a fitted slope. Its interval resamples problems and so carries task-sampling variance. The statistic itself is made of reading-sampling variance, which the interval does not describe. The bar should therefore be read as a statement about these 17 instances, not about a population slope.

The constrained fit puts the asymptote at 31.5%, but on our own registered criterion that figure is an extrapolation, and the raw union at eight readings is the number to quote. What the data support is diminishing returns at the budget we could afford, not saturation: at 1.9 points per reading the curve was still climbing when we stopped. The bar also depends on

where a family was stopped. Exact subset-averaged increments do not grow with K (this curve's is 2.9 points at the fourth reading and 1.9 at the eighth), so a fixed bar is harder to meet at $K = 4$ than at $K = 8$, and meeting it at either says little by itself about where the curve ends.

Twenty readings pooled across three auditor families reach 48.2% [36.7, 60.0], and the same twenty readings flag 54 of 150 correct increments, 36.0% [28.2, 44.2]. That is more than twice the shipped route's 16.0%, so pooling buys its coverage at its own price. Those twenty readings are also not twenty opportunities. The pool is eight cross-vendor, eight same-vendor and four `astra` readings, and the same-vendor route gains 1.8 points from its first reading to its eighth against the cross-vendor route's 19.3. It is sent temperature 0 and returns nearly the same verdict each time, though this design cannot say how much of that is the temperature and how much the model. Counting that route as roughly one reading puts the depth nearer thirteen than twenty (a heuristic, not an estimate), and the pooled figure should be read as a union over three routes rather than over twenty independent looks.

The price is measured on the same footing. Union false positives on 150 correct increments reach 16.0% [10.1, 22.3] at eight readings. We also compute the recall bought per point of false positive: recall gained over one reading divided by false positives gained over one reading. It is 1.68 at eight readings and changes little as readings accumulate. From $K = 2$ to $K = 8$ the ratio runs 1.67, 1.63, 1.64, 1.65, 1.66, 1.67, 1.68. It falls once, at $K = 3$, and then rises, so repeated reading on this route buys recall at a slightly improving price. The ratio is a quotient of two measured rates and carries no interval of its own.

G.2. Which model reads, and under which severity rule

Replacing the auditor with a stronger model of the generator's own vendor moved union recall at eight readings by -26.4 points, cluster $[-37.3, -15.6]$: 4 of 110 against 33 of 110. Three instances were flagged only by the new family and thirty-two only by the shipped auditor, at an exact McNemar p of 4.2×10^{-7} and a cluster sign-flip p of 2.0×10^{-5}. This was the auditor-family study's single preregistered primary, so no family correction applies. Three measurements qualify what it means, and none of them favours reading it as a statement about auditing ability.

First, the two auditors are not at the same operating point, and this paper elsewhere warns against comparing auditors at different operating points. The stronger model flags correct code at 3.3%, or 5 of 150, which §F.3 says to quote as a count. Its bootstrap interval, [0.7, 6.7], sits in the range where simulations of an analogous interval cover about 0.89 at a true rate of 0.03 rather than 0.95, so it may be narrower than it claims. The shipped auditor flags 16.0% [10.1, 22.3]. The stronger model blocks less of everything.

Second, the sign reverses under a weaker severity rule (Figure 6). That rule counts any finding rather than only findings graded BLOCKER; it is exploratory and was not preregistered. Under it the stronger model reaches 59.1% [47.3, 70.6] on the defect population at 34.0% [26.5, 41.7] on correct code, and the shipped auditor reaches 34.5% [23.6, 45.5] at 19.3% [12.8, 26.3]. The ratio of recall to false positives, a derived point ratio with no interval, is 1.74 for the stronger model and 1.79 for the shipped auditor, so nothing here shows one family buying more recall per false positive than the other. Under any-finding grading the stronger model's findings cover more instances, and under the BLOCKER rule fewer, so the -26.4 depends on the counting rule. That does not show why the families' findings differ, or that any finding identifies the hidden failure.

Third, the two families are not sampled alike. The shipped cross-vendor route's capability card does not permit a temperature, so no sampling parameter is sent and the route varies between readings; the same-vendor route permits one, the layer sends temperature 0, and that route is nearly deterministic. A union over nearly identical readings cannot grow much, and it does not: over $K = 1$ to 8 the cross-vendor family gains 19.3 points and the stronger same-vendor family gains 3.0. At one reading the gap between them is -10.1 points, not -26.4. The rest accumulates as the two unions grow at different rates, and because model and sampling differ together, the design cannot say how much of that difference is the sampling and how much the model. We regard this as the most serious limitation of the contrast, and §7 records what would be needed to remove it.

A third family points the other way and belongs beside the headline. A high-reasoning model of the cross-vendor family reaches 32.7% [20.7, 45.0] union recall at four readings while flagging 10.7% [5.9, 16.1] of correct code, against the shipped auditor's 30.0% at 16.0% with eight. Paired on the same instances, the two advantages are never established together. At a common four readings it leads on recall, $+11.5$ [2.9, 21.3], with an unresolved false-positive difference of -0.3 $[-3.8, +3.4]$; against the shipped auditor's full eight it leads on false positives, -5.3 $[-10.3, -0.7]$ unadjusted, but

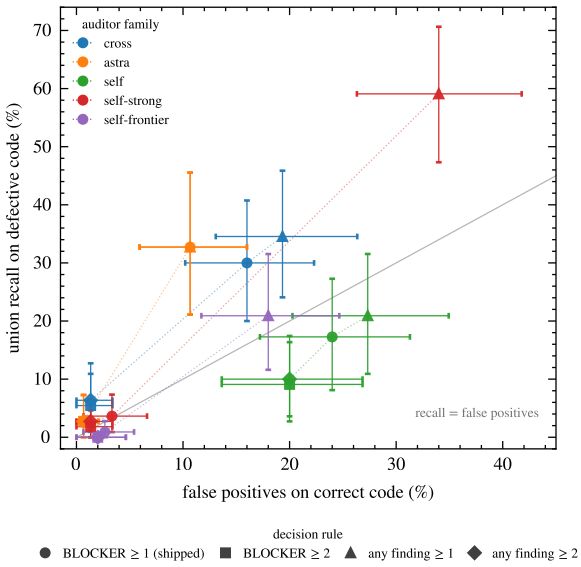


Figure 6. **Union recall is a property of the decision rule, not only of the auditor.** Each point is one auditor family re-graded under one of four decision rules on the same archived readings, with problem-cluster 95% intervals on both axes; no new model calls were made. The families are read at different depths: `cross`, `self` and `self-strong` at all eight of their readings, `astra` and `self-frontier` at their four. The figure therefore places each family at its full archived depth and is not an equal-depth comparison; the common-depth contrasts are in Table 3. The dotted guides join a family's four points and are not a measured curve. Two things Table 3 cannot show: the families do not lie on one frontier, and `self-strong` has no point under the four deterministic rules between 3.3% and 34.0% false positives, exactly where a comparison matched to the cross-vendor auditor would sit, so its ranking there is bracketed rather than measured. Nothing in this figure assesses finding content; a flag is not a diagnosis.

not on recall, $+2.7$ $[-6.5, +13.0]$ (§G.5). The evidence therefore does not support "a stronger model is a worse auditor". It supports the narrower statement that this particular same-vendor model, under the preregistered BLOCKER rule and sampled at temperature 0, blocks far less.

A same-vendor route of the generator's own size reached lower union recall than the cross-vendor one on these instances, -12.7 points $[-25.0, -0.9]$ at eight readings, signed so that a negative value favours the cross-vendor route. The interval is unadjusted, its upper end is 0.9 points from zero, and that route is sent temperature 0, so this is a direction under this sampling and not a ranking of auditors. A study on a second substrate was meant to test whether the sign reverses there; it closed without a quotable number, and on its clean run the same contrast included zero, so no reversal was established either (§G.5). The figure above is a measurement on this substrate, and whether it is a property of same-vendor auditing is open.

G.3. One rulebook sentence moves flags, and does not measurably move outcomes

Adding a single rule to the auditor's constitution, telling it to find what the visible tests do not cover, moved flags on the defect population from 10 of 56 to 25 of 56: $+26.8$ points, cluster [13.6, 40.4], exact McNemar $p = 0.0003$, cluster sign-flip $p = 0.0009$. Together with the pooled defect-and-correct flag contrast, this is one of only two comparisons that clear the Bonferroni threshold, and the two are the same effect seen twice.

The estimand is narrower than that of the sections above and must be read at its own scale. It is measured on 56 instances from 41 problems, not the 110-instance defect population. It is recall at a single reading, union recall at $K = 1$ on that population, because every loop arm revises once. And it is exploratory, because the preregistration named outcome contrasts rather than flag contrasts. For those reasons we do not compare its magnitude with the -26.4 or the $+19.3$ of the preceding sections, which are union-recall differences at $K = 8$ over 110 instances (a contrast between routes and a gain from $K = 1$).

Two costs travel with it. On correct code the same rule moved flags by $+12.50$ points. Every discordant pair points the same way (7 against 0), so by the rule stated in §F.3 the percentile bootstrap's bound at zero is an artefact, and the admissible interval is the unconditional one. That interval is $[-3.6, +26.8]$ and does not exclude zero. It is approximate rather than guaranteed exact, because it uses a grid-approximated nuisance supremum and ignores clustering. Its exact McNemar and

cluster sign-flip p are both 0.0156, against that study's Bonferroni threshold of 0.00313. The directed statement the records support is the pooled one, where 23 discordant pairs point one way and 1 the other (exact McNemar $p = 3.0 \times 10^{-6}$, cluster sign-flip $p = 1.0 \times 10^{-5}$): the sentence makes the auditor louder overall. On the correct stratum alone the cost is $+12.50$ points with an interval that contains zero.

The second cost is to outcomes. The contrast that isolates the rule compares the arm with the added rule against the cross-vendor arm on hidden-suite pass rate after one revision, over both strata (112 instances on 96 problems). The effect is $+5.36$ points, cluster $[-0.89, +12.07]$, exact McNemar $p = 0.146$ and cluster sign-flip $p = 0.183$; the interval does not exclude zero. The rule demonstrably moves what the auditor writes down. Whether that converts into code that passes the hidden suite is not established, and since this paper argues that a flag is not a catch, that distinction applies to our own intervention first.

G.4. Most of what the auditor missed is failure the specification never determined

Fifty-seven of the 110 defective instances were flagged by no reading of the three families read at that point. That is a statement about twenty readings, not about detectability. If an instance is caught with probability p on each independent reading, twenty readings miss it with probability $(1 - p)^{20}$, which is still 98% at $p = 0.001$ and 36% at $p = 0.05$. Never flagged here does not mean never flaggable, and we use the residual as a population to characterise rather than as a set of defects shown to be invisible.

We can put a number on how flaggable it is. Reading six families at forty readings (five distinct models), including the same auditor under a rulebook with one sentence added, shrinks the never-flagged set from 57 to 32 of 110, and 43 of the 68 instances both raters call specification-undetermined are flagged by some reading of some family. Being undetermined is not sufficient for being missed. What does not move is the composition: the undetermined share of the never-flagged set is 44 of 57, 77.2%, at three families and twenty readings, and 25 of 32, 78.1%, at six families and forty readings.

The first classification of that residual asked whether the visible tests exercised the failing input before it asked anything else, and it called 46 of 57 unexercised edges. Re-rating the same instances under a second protocol reverses that. The second protocol asks first whether the specification's prose determines the hidden suite's expected value, redefines the categories by entailment, and uses two raters in consensus; ordering, definitions and raters all changed together, and the study cannot isolate which mattered. The two raters agree that 44 of 57 (77.2%, cluster [62.1, 91.1]) are cases where the prose does not settle the value, and that 2 of 57 are cases where it does and the visible tests merely never built the input. The preregistered condition for withdrawing "the residual is mostly unexercised edge" fires on both of its clauses, and that claim is withdrawn. The interval is conditional on a category and has no committed coverage simulation, so it is uncalibrated, as is every category-conditioned interval we report.

A classification that reverses when its protocol changes is, by itself, weak evidence about the auditor, and the raters are not independent of it: one was the agent that ran this programme (§F.4), which wrote the rubric and knew the earlier labels and which claim was at stake. Three things support the second reading. The rubric was fixed before any label was written. The labels were committed before we looked for outside evidence. And when a first pass was found to have carried instance identifiers and a sheet name into the raters' view, the labelling was re-run with identities stripped and the result held. An outside party had also independently recorded specifications as defective, in twelve MBPP tasks named in a paper written for another purpose (Richter & Papadakis, 2026). Of the 11 that fall inside our defect population, our raters agree on 9, and both disagreements lie on the boundary that paper's own "incomplete" category draws. That check is one-sided by construction: it can support agreement where an outsider looked, and can bound nothing about the rate.

A rater outside the residual's families, asked a different question. One of the two raters above is `gpt-6-astra`, whose own misses help define the residual. A model that is not one of the families defining it, `gpt-5.6-luna` (which read these instances only as a secondary audit route in an earlier architecture study), later rated the same 57 instances post hoc under a different rubric: whether the prose determines an answer at all, on a sheet showing at most three failing inputs and never the expected values (§G.5). It labels 38 of 57 undetermined, 13 determined and 6 cannot-tell. The two figures measure different constructs. The earlier category also admits prose that determines an answer the oracle contradicts, which this rubric does not count as undetermined, so 38 is not a corrected estimate of 44. What they share is that a participating and a non-participating rater both put a majority of the residual on the undetermined side. The non-participating rater agreed with itself on only 7 of 11 repeated items.

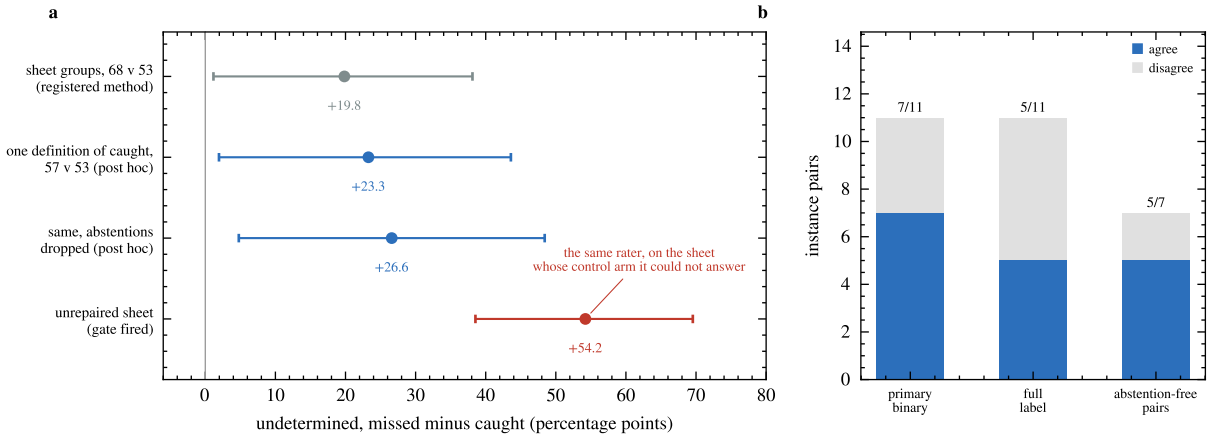


Figure 7. **The residual contrast, printed beside the instrument that produced it.** a, The share of instances a rater judged specification-undetermined, missed group minus caught group, under each reading, with problem-cluster 95% intervals. The two sheet groups use different definitions of *caught* and share eleven instances, so the two readings that apply one definition are the ones that compare populations; both are post hoc. The red point is the same rater on the same sheet before its control arm was rebuilt, when 42 of 53 caught entries carried no failing-input evidence and all 42 drew `cannot-tell`. With that arm the contrast was larger and pointed the way the hypothesis points. The repair and a fresh model run changed together, so presentation, sampling and batch context are not separated, and a preregistered one-third `cannot-tell` gate excluded the earlier figure. b, The same rater's agreement with itself on the eleven instances the sheet carries twice, with identical text and identical evidence. The contrast in a is read off the instrument in b, and the claim this paper draws from it is stated under that constraint.

G.5. Nine further studies, and what they license

Eight further studies cleared independent review after between three and eleven rounds each, and a ninth, on a second substrate, closed without a quotable number. We report them here rather than folding them into the sections above because several are narrow or negative results whose limits are part of the claim, and because the shape of their refusals is itself a finding about this method.

A rulebook sentence moves a second vendor's family. The referent rule, *find what the visible tests do not cover*, raises union BLOCKER recall on `self-strong` (`claude-sonnet-4-6`) at four readings by +20.9 points, cluster [10.8, 31.8], at a false-positive cost on correct code of +8.0 points [3.4, 12.7]. This reproduces §G.3's effect on a different vendor's model at a different K, with its own price. A second intervention, a rule on how findings are graded, shows no demonstrated improvement: $+0.9$ points $[-1.8, +4.5]$. That hypothesis fails its registered criterion, and the failure demonstrates neither equivalence nor that the rule did nothing. The same study also measured, post hoc, whether a flagged finding *reports a failure* on the instance's failing input class, that is, says that class raises or returns the wrong value. The count is 3 of 110 (2.7% [0.0, 7.3]) under the shipped constitution and 27 of 110 (24.5% [14.4, 35.7]) under the referent rule (not the grading rule), and it may not be quoted as defect recognition. Its rubric admits a finding even where it grades the class non-blocking or says the specification is silent, so findings that disclaim the requirement are counted in.

The added rule raises union recall to 60.9%; a higher ceiling is not established. On the defect population the referent rule takes union recall from 33 of 110 to 67 of 110, or 60.9% [48.6, 72.5]. This is the same quantity as the shipped auditor's eight-reading headline in §G.1: blocked by at least one BLOCKER, unioned over eight readings, here for the shipped auditor with the referent rule against the shipped auditor alone. The gain is +30.9 points [19.1, 43.1], at a cost of +20.7 points [12.8, 28.8] in false positives. The single-draw false-positive rate is 23.6% [17.1, 30.3] and fails the product bar on all eight draws. The caution that a flag is not a catch attaches to both figures equally. What separates them is their price, 36.7% [28.5, 44.8] false positives against 16.0%, which is why the shipped route is the shipped route. The fitted asymptotes differ by $+27.1$ points [10.9, 39.0], and *that does not establish a higher ceiling*. The cross-vendor arm never flattened, so one of the two terms is an extrapolation. Heterogeneous low-probability detection imitates a ceiling at this budget, since an instance found with probability 0.02 per reading survives eight readings about 85% of the time. And this study's own zero-inflated estimator finds no never-detectable class in *either* arm. The study's adjudication kill is arithmetic rather than a labelling artefact. Only 9 of 110 instances drew any finding on the single reading whose finding texts were archived, so the largest defect-asserting count available under perfect adjudication was 9, below the floor of 20. Every adjudication-derived figure in the study is conditional on labels whose production can no longer be audited. One rater's raw reply, prompt, launcher

and execution log lived only in a session scratchpad, and they were destroyed between the review that flagged them and the repair.

Corroboration does not validate a generated test. A wrong generated test, one that fails on the dataset's canonical solution, is not reliably recognised by asking whether further independent generation draws also fail on the same candidate. Neither rule that makes further generation calls reaches the preregistered threshold on the registered population. That population covers failing applications on all three candidate strata; 58 of its 107 failing applications are on candidates that already fail their visible tests. The retained wrong-test rates are 12.2% [3.6, 22.6] (Wilson [7.0, 20.6]) for majority-over-draws and 14.9% [6.5, 25.5] for any-draw agreement, against a limit of 2%. Neither is shown to improve on the free within-draw comparator at 12.8% [3.9, 23.9]: the better of the two rules keeps the same wrong applications plus four correct ones, a small observed advantage the study cannot resolve. A post hoc analysis restricted to the candidates a product would meet, those passing their visible tests, gives 6 of 33, 18.2% [0.0, 44.8], and 10 of 43, 23.3% [6.8, 47.1]. The first leaves its relation to the 2% threshold unresolved; the second's unadjusted interval lies wholly above it, which is exploratory evidence of exceeding it. Neither is a preregistered test on that population, whose registered kill rule was a point estimate on the mixed population. A floor of 2.4% holds on the original unique-test quantity because 29 of the 45 wrong tests never fail on any candidate at all. The claim is negative and bound to this substrate: these rules do not validate generated tests here, which is not to say generated tests cannot be validated.

An injected population does not give the residual result a prospective test. We constructed a population of injected defects intended to make specification-determinedness hold by construction. It does not. The filter registered to recover each edit's first failing hidden input is a truthiness check on a field the injector wrote: of 281 recorded witnesses, 179 are strings and 102 are lists, and none is executed. Four of the remaining five filters accept cases their registered wording excludes, such as a suite that reports success after an early exit, a process that terminates without an assertion or an exception, and an added import from a module already imported. No filter establishes specification entailment, and repairing the first one alone would not have: recovering the failing input and executing a witness shows that the edit changes behaviour on an input the hidden suite exercises, not that the specification entails the value expected there. What may be quoted is descriptive. At eight readings the auditor blocked 90 of 92 accepted injected instances and 11 of their 92 unmodified twins, with 79 discordant pairs one way and none the other. On the natural residual it blocked 33 of 110. The instances the construction's own gate *refused* were blocked 31 of 40, which is why that gate cannot be assumed conservative. A probe separates the injected set from the natural one at 96.7% on the items it answered, *by a feature it does not identify*: the two arms share no problem, so task composition alone could produce the result, and the probe's model is one of the two gate models. §G.4 therefore keeps exactly the status its own study gave it, post hoc and unreplicated. A successor needs an independent party to validate the actual violation *and* the specification-entailing expected behaviour under a fixed, blinded protocol. Neither a second substrate nor an independent rater suffices alone, because task composition and construction confounding survive both.

A study on a second substrate closed without a quotable number. In a second benchmark, the visible-test text shown to the auditor *and* to the generator was sliced out of its test class without the class header, so it failed to parse on all 300 tasks while scoring executed the intact class. The repair for that fault introduced a second one, re-attaching a hidden method's decorators to the first visible method on 26 tasks. A third run on a clean extraction reproduced five of the six reversals of first-substrate findings that the second run had produced, and did not reproduce the sixth. The criterion registered before those numbers existed was stated over the set, so the strict reading applies: the study is closed after three runs and about $82, with no quotable number. Whether five of six should have licensed the five is recorded as an open question in `CLAIMS.md`.

Measured union recall is rule-dependent, and a headline's magnitude belongs to its own rule. We re-graded findings already archived by two of the studies above under four decision rules, with no new model calls. The shipped BLOCKER threshold's estimate stays intact for its own rule, which *was* preregistered as "≥ 1 BLOCKER finding", and *reverses its sign* under any-finding grading. That reversal is a mechanical consequence of changing what counts, and it identifies nothing about why the families produced different findings or different severities. At a common depth of four readings, `astra` exceeds the cross-vendor auditor on union recall over the defect stratum by +11.5 points [2.9, 21.3], paired within instance, with a false-positive difference of -0.3 points $[-3.8, +3.4]$. The two sides are not averaged alike. The cross-vendor arm has eight readings and is averaged over all $\binom{8}{4} = 70$ four-draw subsets, while `astra` has exactly four, so $\binom{4}{4} = 1$ and its estimate is a single realisation with none of its reading-selection variance averaged out; one arm's luck in four draws is fully in the number and the other's is not. On the cross-vendor auditor's complete eight-draw ladder the same contrast is $+2.7$ points $[-6.5, +13.0]$ with a false-positive difference of -5.3 points $[-10.3, -0.7]$. That comparison points toward lower false positives, not higher recall. The false-positive interval's upper end is 0.7 points from zero, and it is not adjusted

for the other contrasts computed beside it, so this is a direction the data lean and not one they establish. Observed false-positive rates near one another are not a matched operating point. One family has no point under the four deterministic rules between 3.3% and 34.0% false positives, precisely where a matched comparison would sit, so its ranking is bracketed rather than measured. Nothing here assesses finding content against the hidden failure, so none of it speaks to what any finding identified.

The ambiguity gap survives a rater that did not write the rubric, and the instrument that shows it disagrees with itself. The contrast in §G.4 between missed and caught instances had been rated by two models, one of them the agent that wrote the rubric. A third model re-rated the whole sheet after we rebuilt it on mechanically recovered witnesses. That rebuild was necessary because the control arm had been unanswerable: 42 of 53 caught entries carried no failing-input evidence, and all 42 drew `cannot-tell`. The share labelled *undetermined* is 43 of 68 against 23 of 53, +19.8 points [1.2, 38.1]. Those two groups are not two populations: they define *caught* differently, and 11 instances fall in both. Removing those 11 from the missed group gives $+23.3$ points [2.0, 43.6], post hoc and larger, so the overlap had understated the contrast. The limit is part of the result. On those 11 instances, shown with identical text and identical evidence, the rater agreed with itself on 7 of 11 on this binary outcome and 5 of 11 on the full label, where the earlier study's two raters each agreed 11 of 11 (Figure 7). Every interval's lower end lies within five points of zero, and this remains a model; no human has rated this material. The *unrepaired* sheet scored $+54.2$ points [38.5, 69.5], 2.7 times the repaired contrast and in the direction that flatters the hypothesis. That is a descriptive association between one repair and one fresh re-rating, not evidence about specifications: the repair, the run and its sampling changed together, and the preregistered one-third `cannot-tell` gate catches it.

Adding the missing rule to the specification moves diagnosis, but only where the rule did not wholly contradict the oracle. The contrast above conditions on the specification; this study changes it. The population is the 44 never-flagged instances that both earlier raters called specification-undetermined. For the 32 on 19 problems that passed the generation gates, a model shown the failing *inputs*, but never their expected values, wrote one behavioural rule into the prose, and an auditor re-read the same code and the same visible tests under the original, the clarified and a same-length placebo specification, four readings each. An instance counts as diagnosed only if both adjudicators judged some finding to state the behaviour the hidden suite expects. Diagnosis rose from 0 of 32 to 9 of 32 under clarification and stayed at 0 of 32 under placebo. The registered primary, clarified against original, is +28.1 points, problem-cluster 95% interval [7.7, 51.4], which meets the registered criterion. The registered secondary uses the 23 instances where the re-rating study's consensus (§G.4) and the third rater's label agree; it is 8 of 23 against 0 of 23, $+34.8$ [10.0, 60.9].

Given the observed data, that criterion added little. Every discordant pair points one way, so no bootstrap resample can fall below zero, and with successes on five of 19 problems a resample is exactly zero only when it misses all five, with probability $(14/19)^{19} \approx 0.003$; the registered run has 27 such resamples in 10,000. Once five success-bearing problems are observed, the interval's exclusion of zero adds little to them (with one such problem it would not have excluded zero), and §F.3 says to read a one-signed contrast from an exact test instead. The registration kept the bootstrap and we report it as registered, but the informative figure is the cluster sign-flip test the registration prefers. The nine successes sit on five problems, and the sign-flip test gives $p = 0.060$ (exactly $2/2^5 = 0.0625$ on enumeration), against 0.0039 for a McNemar test that counts nine independent pieces of evidence where five exist. It is not significant at 0.05. The registered naming-rate secondary was never computed.

Two further limits are part of the result. *The inferred rule contradicts the hidden suite entirely on 23 of the 32 instances and partly on 2*, and diagnosis was 0 of 23 where it contradicted against 9 of 9 elsewhere. That split was classified after the outcome was seen, not assigned, and the agreeing instances may simply be the easier problems; it does not say which property of the edits moved diagnosis. A separate model rated the clarified prose more determining than the original by $+28.1$ points [2.6, 54.3], but more determining than the placebo by $+18.8$ $[-9.4, +45.2]$, so whether the placebo was inert as a specification is unresolved. One adjudicator is the agent that ran the study and the other is `gpt-6-astra`, the model that also reviewed this study (§F.4); they agreed on 84 of 91 items, $\kappa = 0.85$, which is agreement and not accuracy. The study supports a specification-edit contrast on these selected instances. It does not establish that underdetermination caused the original misses, and §G.4 is not strengthened into that claim by it.

What a flag on correct code points at. The false-positive rates above count flags on code that passes its hidden suite. A suite may leave behaviour unexercised, so a flag there need not be a false alarm. A preregistered study re-read the 150 correct instances with the shipped route at eight readings, keeping the finding texts the ceiling study had discarded; 20 were flagged (13.3% [7.9, 19.2], a fresh sample of the route's 16.0%). A model given each finding, but not the reference solution

or the hidden tests, extracted the concrete inputs it names, and candidate and reference were run on them. Two of the 20 disagreed with the reference on an extracted input the hidden suite does not exercise (10.0%, problem-cluster [0.0, 26.3]); on nine they agreed, and nine yielded no valid input. Because the interval includes zero, the registered reading is that at this resolution there is no evidence that this route's flags on test-passing code identify reference disagreements. No class licenses the words "true positive" or "false alarm": a disagreement does not show that the specification was violated, and agreement on an extracted input does not show a finding wrong.

What the refusals were about. These eight studies cleared review at rounds 3, 3, 4, 6, 7, 9, 10 and 11, fifty-three rounds in all. Most of those rounds ended in a refusal of the *repair* rather than of the measurement. Measurements moved on review in three of the eight: the sweep's first review found its loader reading one family's draws 4 to 8 as 1 to 5 and another's population from a partial draw, and every cross-family figure was recomputed; a label count in the third rating's report was 71 where its data say 67; and the clarification study's primary moved by one instance and three classification rows (§F.4). Every figure above reproduced independently at every round after its last repair. The recurring failure was a check or a sentence that claimed more than it delivered. A binding protected one sentence of a withdrawal spread across six; when it was rewritten to protect all six, two of them were truncated before the words that carried their meaning. One check stripped HTML comments and called the result what a reader sees, and one test was named for a code path that never called it. A second pattern appeared in the last thirteen rounds: most of those refusals were *overstatements away from our own interest*, such as a registration whose chronology its own repository contradicted, a withdrawal that overcorrected past what the record supported, a publication prohibition a registered gate never imposed, and a limitation credited to another study's work that the other study had not performed. Understatement was corrected as often as overstatement; in both cases the fix was a smaller claim written where the claim was.

G.6. What follows for a recall number

Taken together, the results above say an audit recall figure is a joint property of the auditor and its material. It moves with which route reads (model and sampling together, which this design does not separate), with what the rulebook tells it to look for, and with the severity threshold at which a finding becomes a block, which is rarely reported. What it misses is associated with whether the hidden suite asks a question the specification answers. That association is observed in the residual and in a specification-edit contrast on selected instances; it is not isolated as a cause.

H. The science studies in full

The general-code studies (Section D) measured the model tier on short general-purpose functions. A scientist would hand an auditor three different jobs: check the code that computes a result, check that a reported result follows from that code, and check the data an analysis starts from. We ran one preregistered study per job, each with ground truth that no model supplies. Scientific code (§E.1) is judged by scientists' numeric tests, results (§E.2) by re-execution, and data (§E.3) by the faults we injected. Every item in all three is audited through the product's own audit call with its shipped rulebook, and the same flag rule holds throughout: an item is flagged when the model grades any finding BLOCKER.

H.1. Scientific code

Substrate. SciCode (Tian et al., 2024) decomposes 80 research problems from 16 subfields into 338 steps, each a function with numeric tests written by scientists. Its targets are released separately from the code, which lets the tests act as a hidden oracle. On the benchmark's development split, the gold code passes 48 of 50 steps in our environment; the two steps whose gold fails here are excluded, as are the three the official harness skips.

Population. The generator of the general-code studies wrote every step in SciCode's own protocol, three samples per step, each sample building on its own earlier steps. A candidate is *correct* if it passes the step's tests and *defective* if it fails or times out. A step enters either stratum only if at least one sample passes it, so a failure counts only where this environment has shown the step can be passed; 184 of 336 evaluated steps had no passing sample and were excluded. This yields 81 defective instances from 30 problems and 374 correct ones, from which 150 (53 problems) were drawn by seed. Labels are per step, but the program each auditor sees holds the sample's functions for every step up to the one tested, and 51 of the 81 defective and 49 of the 150 correct instances have a failing earlier step in their chain (post hoc description).

What differs from the general-code studies. SciCode has no visible and hidden split, and we did not invent one. The auditor sees the problem, the step, its background, the function header, the allowed dependencies and the candidate, and

no tests of any kind. The shipped cross-vendor auditor and the generator's own model each read every instance eight times. We reach the generator's model through a command-line route rather than the API. The route adds a short fixed context and uses default sampling, so this study's same-vendor family is not temperature-0, unlike the one in the general-code studies.

Two arms. After the registered arm's outcomes, a second registered arm changed only the task text, naming the step's function as the deliverable; the program bytes are identical.

Residual. Defective instances flagged by no reading of either family are rated on a blind sheet under the general-code residual's question (§G.4) adapted to science: do the step's prose and background determine the value the test expects, including tolerance, units, conventions and numerical method. The raters are the agent that ran the study and a model that audits nothing here.

Results. In these runs, how the task framed the deliverable changed the flag rate substantially (Figure 5a). With the whole problem as the task, the shipped auditor flagged 87.7% [72.7, 98.6] of defective and 68.7% [56.6, 79.7] of correct instances at eight readings; some of its objections to correct code were that one step did not implement the whole problem. With the task reframed around the step, flags on correct code fell to 36.0% [26.4, 45.6], a paired change of -32.7 points [-42.8, -22.4]. Recall fell too, to 72.8% [59.3, 87.3], a change of -14.8 points [-28.6, -2.7], though the cluster sign-flip test for that recall change gives $p = 0.0625$. The change is the whole reframing (the step named, the problem description moved last and marked as context, earlier functions exempted), and the arms ran in sequence. The recall of the generator's own model was 46.9 points below the shipped auditor's in each arm ([-63.0, -30.7] and [-61.5, -34.4]; a route comparison). It flagged 27.3% [18.4, 37.3] and 18.0% [11.5, 25.6] of correct code in the two arms, respectively. At one reading in the reframed arm the shipped auditor flags 61.7% [47.9, 75.9] of defective against 19.8% [12.9, 27.2] of correct instances: it discriminates, modestly. We read a seeded sample of 20 flagged correct instances whose earlier steps all pass, taking the first BLOCKER of the first flagged reading of each; four of the twenty concern earlier functions that the reframed task excluded from the deliverable. They range from real divergences between code and step text, through contradictions inside the benchmark's own instructions, to tests too weak to separate outputs; which flags point at defects was not established. An independent expert audit of SciCode's test problems found self-contradictory specifications, non-reproducible gold answers and over-tight tolerances that reject correct solutions (Hu et al., 2026), so some instances labelled defective here may be correct code the benchmark rejects. Of the 81 defective instances, 7 and 19 escaped all sixteen readings of the two families in the two arms. Both raters judged 4 of 7 (two problems; cluster interval [0.0, 80.0]; $\kappa = -0.235$) and 13 of 19 (eight problems; [28.6, 93.5]; $\kappa = 0.587$) undetermined by the prose, too few problems and too little agreement for a stable share. The registered test that this share exceeds the general-code residual's 77.2% is killed in both arms by its own interval rule. The rating sheets showed only the step's function, not helpers its test calls, and truncated long expected arrays; of six items that appeared on both sheets, the second rater changed three labels.

H.2. Scientific results

Items. Each correct SciCode instance whose output on one seeded test case is a real scalar or a real array of at most ten elements becomes a small results increment: the program, the test's inputs, a script that runs one on the other and writes a run log, a `results.json` citing each value's log line, metadata declaring every input at a revision, and a one-paragraph report whose numbers carry the product's number→source annotations. Seventy-four of the 150 correct instances qualify, from 36 problems. Each yields one clean item and one faulty item. The faulty item carries one of seven faults, assigned in rotation by seed. Five are *reporting* faults, where the artefacts disagree with each other or with the log the code wrote: a $\times 10^3$ scale slip, a sign flip, a value from another test case, a 5% relative error, and a transposed digit pair in the report alone. Two are *fabrications*, where the scale slip or the 5% error is written consistently into the log, the results and the report, so that every artefact agrees with every other and none agrees with the code.

Three auditors. The LLM auditor reads the increment four times, with the project's checks set to the shipped `science` profile. The model's prompt includes the deterministic result, as it does in the product, so on reporting faults the model has seen the deterministic finding. The deterministic profile's own verdict comes from the same call. Re-execution runs the program on the inputs and compares the output with the reported values.

Gate. Before any model call, every clean item had to pass the science profile with no BLOCKER and re-execute to its reported value; all 74 did when first built. A two-reading pilot then found an instrument defect: no file in the item performed the run the report described, and the auditor blocked on that, correctly. We added the run script, discarded the pilot readings, and added a third condition to the gate, that each clean item's run script reproduce its run log byte for byte. All 74 passed all three conditions with no repair.

Results. The deterministic profile does what it was built for (Figure 5b). It flags all 49 reporting faults, because the log, the results and the report no longer agree and number→source says where, and none of the 25 fabrications, because in those everything agrees with everything else. It flags no clean item. The LLM auditor flags 23 of the 25 fabrications at four readings, 21 of them with a sound finding about the fault (post hoc). Those findings work the result out from the code and inputs, for example evaluating a KL divergence of two small distributions to 1 where the report says 1000. The other two rest on calculations that are themselves wrong. Paired on the faulty items, all 23 discordant items go the LLM's way (exact McNemar $p = 2.4 \times 10^{-7}$). A problem-level sign-flip test, run post hoc, gives 3.8×10^{-6}. On the reporting faults its rate is of the combined workflow, since its prompt carried the profile's findings. Re-execution flags every fault except four whose change to a near-zero output falls within `np.allclose`'s default absolute tolerance; two of those outputs are floating-point zeros. Setting those four aside (post hoc), re-execution flags 70 of 70 faulty items at 0 of 70 clean, against the model auditor's 68 of 70 at 12 of 70. On the registered population the two flag 70 and 72 of 74 faulty items, and eight of the model's twelve clean-item flags identify real construction defects, so this subset favours re-execution for injected numerical faults without making it the better check in general.

The LLM also flags 12 of the 74 clean items. We read each of these flags (post hoc), and 8 are right. In 9 instances the benchmark passes the test's inputs inside the call itself, so the report's sentence that the program ran "on the inputs in `inputs.py`" was false; 7 of those clean items were flagged for it. Of four outputs whose unit the solution documents, two were flagged for our `dimensionless` label; one of the two is also among the seven. The remaining 4 flags rest on wrong calculations or, in one case, on a syntax error that is not there. On this evidence a provenance chain and a reader that recomputes are complements: the first is exact about agreement and blind to consistent error, and the second sees consistent error but is sometimes wrong itself.

H.3. Scientific data

Substrate. We use four physical-science tables from the UCI repository, each under CC BY 4.0: concrete compressive strength (Yeh, 1998), airfoil self-noise (Brooks et al., 1989), combined-cycle power-plant output (Tüfekci, 2014), and superconductor critical temperature (Hamidieh, 2018). Exact duplicate rows and physically impossible rows (relative humidity above 100%) were removed from each pool before any item was drawn.

Items. Each item is 120 rows sampled from one pool with a data card that gives every column's meaning and unit and, where the source states one, its range. There are 35 clean items per dataset and five per dataset for each of seven faults: mixed units in 20% of one column's rows, three negated values of a non-negative quantity, 12 duplicated rows, the target shuffled among 30% of rows, two columns with disjoint ranges swapped, a -999 sentinel in 5% of one column's cells, and a continuous column rounded to integers.

Auditors. Both model families read each item four times, with the dataset as the deliverable and the card as its documentation. The comparison is a deterministic validator written from the card alone and frozen before any item was built: declared ranges, non-negativity, duplicates, sentinels and integer-valued continuous columns. By construction it catches the negations, duplicates, sentinels and rounding. It sees mixed units or swapped columns only when they push a value below zero or out of a declared range, or make a continuous column integer-valued, and a shuffled target not at all. The question is whether the model catches what the validator cannot. Two instrument defects were found and fixed by amendment before the readings reported here: the card and file named one column differently, and the files first sat outside the audited scope, so the product had told the model there was nothing to audit.

Results. The two tiers flag overlapping but different faulty items (Figure 5c). The validator flags 92 of 140 faulty items (65.7% [57.9, 73.6]) and no clean item. The shipped auditor flags 65 of 140 (46.4% [38.6, 54.3]) at 4 of 140 clean items (2.9% [0.7, 5.7]), and their union flags 112 of 140 (80.0% [73.6, 86.4]) at the same 4. These intervals resample items within dataset; four datasets are too few to resample, so no interval here carries between-dataset variation. Of the 20 faulty items the auditor flags and the validator misses, 18 carry a finding about the injected fault itself, 9 of them mixed units and 9 swapped columns (post hoc). In one, the auditor reasoned that three atoms spanning a 135 pm radius range cannot have a mean radius of 1.6 pm, which is the cell the fault had converted to ångströms. The other 2, and all 4 flags on clean items, are requests for fuller column descriptions on one dataset's card; so are 2 more of its faulty-item flags, which the validator also caught. Counting only findings about the injected fault (post hoc), the auditor flags 61 of 140 and the union 110, at no clean item. The auditor flagged no shuffled target and no duplicated rows (0 of 20 each), and it did worse than the validator on every fault the validator was written to catch. The generator's own model, read through its command-line route, is not a usable data auditor here: it flags 67 of 140 clean items, and in at least 48 of those one finding states a row count, field count,

missing column or truncation that the file contradicts (post hoc lower bound).

The deterministic validator is not neutral. It was frozen from the card before any item existed, but it omits one range the concrete source's documentation states and uses a list of continuous columns the card does not give; with the omitted range it would flag 93 of 140. Unit conversion also left floating-point tails that mark some converted cells in two of the tables. None of the auditor's mixed-unit findings cites them, but that does not show they had no influence.

Held-out naming of faults at a fixed false-positive budget. A fourth preregistered study (registered before any of its items was built or read) asks what the study above cannot: whether the model auditor names faults the validator misses, rather than flags items, at a false-positive budget fixed before the data exist. It builds 280 new items by the construction above, unchanged except for the seed (no CSV is shared with the development items), and reads each four times with both families. The registered endpoint is scored by code from the injection manifest: a faulty item counts when a flagging reading's BLOCKER text names the injected column, both columns for a swap, or a duplication term for duplicated rows, using a frozen alias table. No model judges any outcome. The rule is lexical; it is not a measure of correct diagnosis, and below we report what it wrongly counts and misses. On the development items it counts 60 of the shipped auditor's findings against the post hoc fault-relevant count of 61. The operating points were fixed on the development items: the smallest vote threshold over four readings whose clean-flag rate was at most 10%, any one reading for the shipped auditor (2.9% on development) and all four for the generator's own model (7.1%), which then counts when at least one of the four flagging readings meets the rule. No threshold of the latter reaches 5%, which is why the budget is 10%.

On the held-out items the validator meets the rule on 90 of 140 faulty items (64.3% [56.4, 72.1]) and flags no clean item. The shipped auditor at its operating point meets it on 57 (40.7% [32.6, 48.9]) at 9 of 140 clean flags (6.4% [2.8, 10.5]), and the generator's own model on 40 (28.6% [21.7, 35.9]) at 14 (10.0% [5.6, 14.8]). The validator with the shipped auditor reaches 105 (75.0% [67.6, 81.9]) at the same 9 clean flags. The registered primary, the union against the validator alone, is $+10.7$ points [6.0, 15.9]: 15 items added and none lost (exact McNemar $p = 6.1 \times 10^{-5}$), at a realised clean-flag rate inside the budget, so the hypothesis holds; the union's interval on that rate reaches 10.5%, so the population rate is not shown to be inside it. The gain comes from mixed units (5 to 13 of 20) and swapped columns (5 to 12 of 20). The secondary route comparison puts the shipped auditor 12.1 points [4.1, 20.3] above the generator's own model (27 against 10 discordant items); model and sampling change together, and the two sit at different realised rates inside one budget. Intervals resample items within dataset. The ledgers value the study's calls at $24.41.

Read against the manifest and item files after the outcome (post hoc), the rule is liberal about meaning and strict about wording. It counts one of the shipped auditor's findings that is a request for column definitions and six of the other model's that are about something else (row counts, the card, missing ranges, near-duplicate records, or a statement that the structure is satisfied), so that model's three shuffled-target and two rounding counts are not diagnoses. It counts one of the fifteen items the union adds, a swap, on a finding that reports the wrong columns; and it misses three findings that describe impossible values in one of the two swapped columns. A stricter post hoc keyword filter, requiring a fault term or a quoted injected value as well, leaves the shipped auditor at 53 and the other model at 32; it is still lexical, and it wrongly drops one swap whose findings describe the two columns as reversed. The four items the shipped auditor loses are validator items, so the union stays at 105. The study agent read the fifteen items the union adds against the manifest and item files, unblinded and after the outcome: fourteen locate the fault in the right columns ($+10.0$ points [5.4, 15.1], exact McNemar $p = 0.00012$), and eleven also explain the injected transformation ($+7.9$ [3.7, 12.5]); three strength items converted to psi are located without the conversion being explained (one finding calls it a decimal shift and one reconstructs a wrong factor). That reading is not an independent adjudication. All nine of the shipped auditor's clean flags are on airfoil items and make one objection, that the card names each column's unit but does not define it; like the four development clean flags, they may be a fair reading of the task sentence rather than errors. The held-out items are new draws from the same four tables and seven synthetic faults, so the result does not extend to other data, faults or cards.

I. Reproducibility and claims

The repository at `https://github.com/dongzhaohe321418-lab/audit-ceiling-paper` carries the claims ledger (`manuscript/CLAIMS.md`), with the claims the evidence does not support; the results report of every study the paper draws on; every registered study's preregistration and numbered amendments (the science studies' under `plan/studies/ai4s/`, where the data and results registrations carry errata); every independent review of those reports, verbatim and including refusals; and the derived records (counts, rater labels, hashes and intervals) that the figures and Table 3 are drawn from. A single command, `reproduce.sh`, regenerates Figures 1–4 and Table 3 from those records and

checks that each is byte-identical to the copy the manuscript compiles, with no model calls, no network access and no path outside the checkout; it was run from a fresh clone with the home directory unavailable to establish that. The repository's `README` maps each claim to its report, its reviews and its records. The corpora are HumanEval (MIT), HumanEval+ and MBPP+ (Apache-2.0), SciCode (Apache-2.0) and four UCI datasets (CC BY 4.0), which permit redistribution. For the three science studies the repository also carries the raw run archive (`records/ai4s/runs/`): the audited items, every reading's verdict and the text of its blocking findings, the rating sheets and the logs, so the post hoc adjudications can be checked finding by finding; the final analysis scripts are in `records/ai4s/analysis/`, and need the harness to run. For the general-code studies it carries each study's derived records; their raw readings regenerate the records deterministically through the analysis code in the auditor harness, a separate public repository named in the provenance file. Without raw readings, records can be regenerated only by re-running the studies against model endpoints whose outputs are not deterministic.